



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

Факултет Приложна математика и информатика

ДИПЛОМНА РАБОТА

**Тема: Платформа за управление и агрегиране на вградими уеб
компоненти за разработване на уеб приложения за туризма**

Дипломант: Аркан Ахмедов Ахмедов

Факултетен номер: 791325012

Специалност: Информатика и софтуерни науки

ОКС: Магистър

Научен ръководител: проф. д-р инж. Малинка Иванова

СОФИЯ

2026

СЪДЪРЖАНИЕ

СЪДЪРЖАНИЕ	2
ВЪВЕДЕНИЕ	5
Глава 1: Постановка на проблема и цел на проекта	7
1.1 Актуалност на проблема.....	7
1.2 Мотивация за разработката	8
1.3 Цел на проекта	8
1.4 Основни задачи.....	8
1.5 Обхват и ограничения.....	9
1.6 Заинтересовани страни	9
1.7 Очаквани резултати	10
Глава 2: Анализ на предметната област и съществуващи решения	11
2.1 Преглед на предметната област	11
2.2 Анализ на съществуващи подходи и системи.....	11
2.3 Сравнение между алтернативните решения.....	12
2.4 Изведени проблеми и възможности за подобрене	13
2.5 Обосновка на избрания подход	13
Глава 3: Изисквания към системата	15
3.1 Функционални изисквания.....	15
3.2 Нефункционални изисквания.....	16
3.2.1 Сигурност.....	16
3.2.2 Производителност	17
3.2.3 Надеждност	17
3.2.4 Разширяемост и поддръжка	17
3.2.5 Съвместимост	17
3.2.6 Удобство за потребителя	17
3.3 Потребителски роли.....	18
3.4 Основни потребителски сценарии.....	18
3.5 Ограничения и допускания	19
Глава 4: Архитектура и проектни решения	20
4.1 Обща архитектурна концепция.....	20
4.2 Избрана архитектурна методология.....	21
4.3 Контекстна диаграма по C4.....	22

4.4 Контейнерна диаграма по C4	23
4.5 Компонентни диаграми по C4	24
4.5.1 Компоненти на каталога и публикациите.....	25
4.5.2 Компоненти на механизма за конектори	26
4.6 Комуникация между компонентите	27
4.7 Модел на данните и база данни	28
4.8 Сигурност и управление на достъпа	30
4.9 Обосновка на архитектурните решения.....	30
Глава 5: Технологии и среди за разработка.....	33
5.1 Преглед на използваните технологии	33
5.2 Клиентска част.....	33
5.3 Сървърна част	34
5.4 База данни	34
5.5 Инфраструктура и DevOps подход.....	34
5.6 Външни услуги и интеграции	35
5.7 Среда за разработка.....	35
5.8 Причини за избора на технологиите	36
Глава 6: Техническа реализация	37
6.1 Структура на проекта.....	37
6.2 Реализация на клиентския слой	38
6.3 Реализация на сървърния слой.....	40
6.4 Реализация на бизнес логиката	41
6.5 Реализация на базата данни.....	42
6.6 Реализация на комуникацията между модулите	43
6.7 Основни алгоритми и процеси.....	44
6.8 Обработка на грешки	44
6.9 Сигурност в реализацията	44
Глава 7: Тестване и валидация.....	46
7.1 Подход към тестването	46
7.2 Модулни тестове	46
7.3 Интеграционни тестове	47
7.4 Проверка на основните потребителски сценарии.....	47
7.5 Тестване на производителност	48

7.6 Анализ на резултатите	49
7.7 Ограничения на проведеното тестване	49
Глава 8: Ръководство за инсталация и използване	50
8.1 Необходими зависимости.....	50
8.2 Стартиране на клиентската част	50
8.3 Стартиране на сървърната част	51
8.4 Конфигурация на базата данни.....	51
8.5 Стартиране с Docker	52
8.6 Основни стъпки за използване.....	52
8.7 Административна част.....	57
Глава 9: Оценка на решението и бъдещо развитие.....	58
9.1 Постигнати резултати	58
9.2 Сравнение с поставените цели.....	58
9.3 Предимства на разработеното решение	59
9.4 Ограничения	60
9.5 Възможности за бъдещо развитие	60
Приложение – Изходен код.....	62
ЗАКЛЮЧЕНИЕ	63
ДЕКЛАРАЦИЯ	64
ДЕКЛАРИРАМ:	64
ИЗПОЛЗВАНА ЛИТЕРАТУРА	65

ВЪВЕДЕНИЕ

Туристическият бизнес в България и в региона се развива чрез дейността на множество агенции и туроператори, които ежедневно предлагат почивки, екскурзии и пакетни пътувания. Обикновено офертите се създават от туроператорите, а се продават от туристическите агенции. Между тези две страни обаче често липсва обща технологична среда, която да свързва създаването на офертите с тяхното представяне пред крайния клиент. В резултат агенциите поддържат съдържанието си ръчно, пренасят информация от таблици и електронна поща и използват остарели начини за вграждане на външно съдържание в собствените си сайтове.

Един от най-разпространените подходи за показване на външни туристически оферти в сайт на агенция все още е вграждането чрез `iframe`. Този метод поставя съдържанието в отделна рамка, която не наследява визуалния стил на страницата, адаптира се трудно към мобилни устройства и предлага ограничени възможности за персонализация. За крайния потребител това често изглежда като външна част от сайта, а агенцията губи контрол върху начина, по който офертите се представят. Така техническото ограничение се превръща и в бизнес проблем, защото отслабва разпознаваемостта на марката и затруднява изграждането на доверие.

Настоящата дипломна работа разглежда проектирането и реализацията на платформа, наречена *Odisea*, която е насочена към решаването на този проблем. Платформата управлява и агрегира споделен каталог от туристически оферти и ги предоставя на агенциите чрез вградими уеб компоненти. За разлика от `iframe` решенията, тези компоненти се изобразяват директно в страницата на агенцията и могат да наследяват нейния визуален облик чрез система от теми, изградена върху CSS променливи. По този начин една и съща оферта може да бъде представена по различен начин в сайтовете на различни агенции, без съдържанието да се дублира и без да се губи възможността за централизирана актуализация.

Платформата обхваща целия път на туристическия продукт: от въвеждането на оферти от туроператорите, през подреждането им в многократно използвани колекции от агенциите, до вграждането им като компоненти в крайните сайтове. Това е реализирано чрез ясно разделение на основните отговорности: споделен каталог, механизъм за филтриране и колекции, система от теми и изгледи, както и слой за разпространение през различни канали. Към тях е добавен и механизъм за конектори, който приема оферти от външни системи на туроператорите по различни протоколи и ги привежда към общ модел.

Темата е актуална, защото пазарът на готови интеграции за туристически оферти все още е силно повлиян от наследени решения,

които разчитат основно на `iframe` вграждания и не предлагат достатъчно гъвкав програмен интерфейс. Платформа, която предоставя реални уеб компоненти, последователна система за тематизиране и ясен модел за интеграция, отговаря на практическа нужда както на агенциите, така и на туроператорите. Освен бизнес стойност, проектът поставя и важни инженерни задачи, свързани с многонаемна архитектура, изолация на данните, тематизиране чрез токени, синхронизация на външни източници и контрол върху актуалността на офертите.

Дипломната работа е организирана в девет глави. Глава 1 представя проблема, мотивацията, целта и задачите на проекта. Глава 2 разглежда предметната област, съществуващите решения и основанията за избрания подход. Глава 3 описва функционалните и нефункционалните изисквания, потребителските роли и основните сценарии. Глава 4 представя архитектурата и проектните решения чрез C4 диаграми и модел на данните. Глава 5 разглежда използваните технологии и среди за разработка. Глава 6 описва техническата реализация по слоеве и включва ключови кодови фрагменти. Глава 7 е посветена на тестването и валидацията. Глава 8 съдържа ръководство за инсталация и използване. Глава 9 оценява постигнатите резултати спрямо поставените цели и очертава възможности за бъдещо развитие. Накрая са представени заключение, приложения, декларация и използвана литература.

Глава 1: Постановка на проблема и цел на проекта

Преди да бъдат разгледани конкретните технически решения, е необходимо да се изясни какъв проблем решава платформата и какви цели трябва да постигне. В тази глава се представят проблемът в туристическия бранш, мотивацията за разработката, целта на проекта, основните задачи, обхватът на системата, заинтересованите страни и очакваните резултати.

1.1 Актуалност на проблема

Туристическите продукти обикновено се създават от туроператорите, но достигат до крайните клиенти чрез туристическите агенции. Туроператорът подготвя пакета, договаря цените и носи оперативната отговорност за услугата, докато агенцията изгражда доверието към клиента чрез собствената си марка и начин на представяне. За да продава успешно оферти на външни доставчици, агенцията трябва да ги показва така, че те да изглеждат като естествена част от нейния сайт. Именно тук възниква основният проблем: липсва удобен технологичен слой, който да свързва системите на туроператорите със сайтовете на агенциите.

В практиката най-често се използват два подхода, като и двата имат съществени ограничения. При първия агенцията пренася офертите ръчно от таблици, ценови листи и електронни писма. По този начин тя запазва контрол върху визуалното представяне, но информацията лесно остарява и поддръжката изисква постоянни усилия. При втория подход съдържанието се вгражда чрез `iframe`. Той решава част от проблема с актуалността, но вградената рамка рядко изглежда като естествена част от сайта на агенцията.

Ограниченията на `iframe` подхода се отразяват пряко и върху потребителското изживяване. Съдържанието в рамката се адаптира по-трудно към различни екрани, ограничава възможностите за персонализация, затруднява оптимизацията за търсачки и усложнява проследяването на поведението на потребителите. В резултат агенцията получава актуално съдържание, но губи част от контрола върху начина, по който клиентът го възприема.

Съвременните уеб технологии позволяват по-гъвкаво решение. Уеб компонентите дават възможност за създаване на многократно използваеми елементи със собствено поведение, а CSS променливите позволяват тези елементи да приемат визуалния стил на сайта, в който са вградени [1]. Така външното съдържание може да остане централизирано и актуално, без да изглежда отделено от страницата на агенцията.

1.2 Мотивация за разработката

Основната мотивация за разработката е да се премахне компромисът между актуалност и визуална идентичност. Агенцията не трябва да избира между ръчно поддържани оферти, които изглеждат добре в нейния сайт, и автоматично заредено съдържание, което изглежда като външен елемент. Необходим е централен каталог, който се обновява автоматично, но се представя чрез компоненти, съобразени с облика и марката на всяка агенция.

Тази идея засяга и доставчиците на оферти. Ако въвеждането и обновяването на съдържание е трудно, каталогът няма да бъде достатъчно полезен. Затова платформата предвижда механизъм за конектори, който приема данни от различни източници и ги привежда към общ модел. По този начин проектът не разглежда само визуалното вграждане, а целия процес - от постъпването на офертата в системата до нейното показване в сайта на агенцията.

От инженерна гледна точка задачата също е подходяща за дипломна разработка. Платформата трябва да обслужва множество независими организации в обща инфраструктура, което изисква изолация на данните, управление на роли и права за достъп, както и стабилна работа при повече потребители. Това налага ясна архитектура, добре разделени отговорности и възможност за проверка на основната бизнес логика чрез тестове.

1.3 Цел на проекта

Целта на дипломната работа е да се проектира и реализира платформа за управление на туристически оферти и тяхното представяне чрез вградими уеб компоненти. Платформата трябва да поддържа споделен каталог, да позволява на агенциите да създават подобрени селекции от оферти и да предоставя тези селекции като компоненти, които могат да се вграждат във външни сайтове и да приемат тяхната визуална идентичност.

За да бъде приложима в реална среда, системата трябва да покрива не само визуалното представяне на офертите, но и тяхното въвеждане, организиране, актуализиране, защита и изолация между отделните организации.

1.4 Основни задачи

За постигане на поставената цел се определят следните основни задачи:

1. Да се анализира предметната област и да се проучат съществуващи подходи за дистрибуция и вграждане на туристически оферти, като се изведат техните силни и слаби страни.

2. Да се дефинират функционалните и нефункционалните изисквания към платформата, потребителските роли и основните сценарии на работа.

3. Да се проектира софтуерна архитектура с ясно разделение на отговорностите и да се обоснове изборът на архитектурен подход.

4. Да се проектира модел на данните, който поддържа многонаемност и проследимост на офертите до техния източник.

5. Да се реализира сървърен слой с програмен интерфейс, автентикация и контрол на достъпа по роли.

6. Да се реализира механизъм за каталог, колекции и филтриране, който поддържа параметризирани селекции от оферти.

7. Да се реализира портал за управление, чрез който агенциите и туроператорите да работят с платформата.

8. Да се реализират вградими компоненти.

9. Да се покрие основната функционалност с автоматизирани тестове и да се осигури непрекъсната интеграция.

10. Да се подготви среда за внедряване чрез контейнеризация и да се изготви ръководство за инсталация и използване.

1.5 Обхват и ограничения

Обхватът на разработката включва платформената част и основните интерфейси, чрез които се работи със системата: публичен сайт за представяне, портал за управление и вградими компоненти за крайните сайтове на агенциите.

Извън обхвата остават някои по-мощни разширения, като пълен резервационен цикъл с документи, автоматично генериране на страници за търсачки и готови приставки за платформи като WordPress и Shopify. Те са разгледани като възможности за бъдещо развитие в Глава 9. Като ограничение се отчита и това, че схемата на офертите е начална версия и следва да бъде допълнително потвърдена спрямо реални изисвания от доставчици.

1.6 Заинтересовани страни

Платформата обслужва няколко групи участници с различни роли и интереси.

Туроператорите са доставчиците на съдържание. За тях е важно офертите да могат да се въвеждат и обновяват удобно, да се запазва проследимостта към собствените им системи и съдържанието им да достига до повече агенции с минимални допълнителни усилия.

Агенциите са разпространителите на офертите. За тях е важно да представят съдържанието в собствения си визуален облик, да изграждат подобрени селекции и да управляват запитванията от клиенти, като запазват контрол върху марката и върху взаимоотношението с туриста.

Разработчиците, които интегрират платформата в съществуващи сайтове, се нуждаят от ясен програмен интерфейс, предвидими компоненти и лесен начин за вграждане без сложна първоначална настройка.

Туристите са крайните потребители. Те възприемат платформата косвено чрез компонентите в сайта на агенцията и очакват бързо, последователно и достъпно потребителско изживяване.

Администраторите на платформата отговарят за общото управление на системата, наблюдението на отделните организации, поддръжката на споделения каталог и правилата за достъп.

1.7 Очаквани резултати

Очакваният резултат е работеща платформа, която покрива пътя от въвеждането на туристическа оферта до нейното показване като вграден компонент в сайт на агенция. Тя включва споделен каталог, механизъм за колекции и филтриране, портал за управление, вградими компоненти с тематизиране, конектори към външни източници и среда за внедряване чрез контейнери.

Очаква се решението да бъде проверено чрез автоматизирани тестове. На концептуално ниво разработката трябва да покаже, че подходът с вградими уеб компоненти може едновременно да запази актуалността на каталога и визуалната идентичност на агенцията.

Следващата глава разглежда в детайл предметната област и съществуващите решения и обосновава защо избраният подход отговаря на описаните цели.

Глава 2: Анализ на предметната област и съществуващи решения

Настоящата глава разглежда начина, по който туристическите оферти достигат от туроператорите до сайтовете на туристическите агенции. Целта не е да се повтаря мотивацията за проекта, а да се анализират съществуващите подходи за разпространение и визуализиране на оферти. На тази основа се извеждат ограниченията, които Odisea трябва да преодолее.

2.1 Преглед на предметната област

Туристическите оферти обикновено се създават от туроператори или външни доставчици и след това се предоставят на агенции, които ги публикуват в своите сайтове. Това поставя агенциите в зависимост от качеството, формата и актуалността на получената информация.

В практиката една оферта рядко е само текстово описание. Тя включва дестинация, период, транспорт, хотел, цена, условия, снимки, наличности и допълнителни услуги. Част от тези данни се променят често, особено цените, датите и наличностите. Поради това ръчното поддържане на оферти е трудно при по-голям обем съдържание.

От гледна точка на туристическата агенция са важни три основни условия. Първо, офертите трябва да бъдат актуални. Второ, те трябва да се вписват визуално в сайта на агенцията. Трето, интеграцията трябва да бъде достатъчно лесна, за да не изисква постоянна техническа намеса. Именно между тези три условия възниква основният практически проблем.

2.2 Анализ на съществуващи подходи и системи

В практиката могат да бъдат разграничени три основни подхода за публикуване на външни туристически оферти в сайт на агенция.

Първият подход е ръчното поддържане на съдържание. Агенцията получава оферти чрез електронна поща, таблици, документи или административни панели и ги публикува сама. Предимството е пълен контрол върху крайния вид на страницата. Недостатъкът е, че информацията лесно остарява, а при голям брой оферти поддръжката става бавна и податлива на грешки.

Вторият подход е вграждането чрез `iframe`. При него съдържанието се зарежда от външна система в рамка, поставена в сайта на агенцията. Това улеснява интеграцията и запазва актуалността на офертите, но ограничава визуалното вписване в сайта. Съдържанието остава отделено от структурата на страницата, което затруднява стилизирането, аналитиката и оптимизацията за търсачки [2].

Третият подход е използването на вградими компоненти и програмен интерфейс. При него данните се управляват централизирано, но се визуализират директно в сайта на агенцията. Това позволява по-добро съчетание между актуалност, контрол върху изгледа и възможност за развитие. Недостатъкът е, че подобен подход изисква по-добре проектирана платформа, стабилен модел на данните и ясни правила за интеграция.

На българския пазар пример за по-стар модел е PeakView. Системата предоставя White Label конфигурации, чрез които агенциите могат да избират оператор, дестинация, транспорт и курорт, след което получават код за вграждане. Налични са и допълнителни възможности като XML интеграции, търсачки по дестинации и услуги за изработка на сайтове.

Този тип решение показва, че за туристическите агенции не е достатъчно просто да получат списък с оферти. Нужни са каталог, партньорска мрежа, настройки за визуализация и механизъм за интеграция. В същото време iframe моделът поставя граници пред по-дълбокото визуално и техническо вписване в сайта на агенцията.

2.3 Сравнение между алтернативните решения

За да се определи подходящата посока за Odisea, трите подхода могат да бъдат сравнени по критерии, които имат практическо значение за туристическите агенции.

Таблица 2.1. Сравнение между подходите за представяне на туристически оферти

Критерий	Ръчно съдържание	iframe вграждане	Компоненти и програмен интерфейс
Актуалност на офертите	Ниска	Висока	Висока
Визуално вписване в сайта	Високо	Ограничено	Високо
Поддръжка при голям обем оферти	Трудна	Лесна	Лесна
Персонализация	Пълна, но ръчна	Ограничена	Управляема чрез теми
Аналитика	Ограничена	Затруднена	По-лесна за интегриране
Оптимизация за търсачки	Възможна	Ограничена	По-добра
Сложност на внедряване	Ниска	Много ниска	Ниска до средна
Контрол за разработчика	Нисък	Нисък	Висок

Ръчното съдържание е подходящо само при малък брой оферти или рядко обновяване. При динамичен каталог то създава риск от неточна информация. `iframe` подходът решава проблема с актуалността, но ограничава качеството на интеграцията. Подходът с компоненти и програмен интерфейс е по-подходящ за платформа, която цели едновременно централизирано управление на оферти и добро визуално вписване в сайтовете на различни агенции.

2.4 Изведени проблеми и възможности за подобрене

От разгледаните подходи могат да бъдат изведени няколко конкретни проблема.

Първият проблем е трудната поддръжка на актуална информация при ръчно публикуване. Туристическите оферти съдържат данни, които се променят често, затова е необходимо централизирано обновяване.

Вторият проблем е ограничената визуална интеграция при `iframe` решенията. Агенцията може да вгради офертите бързо, но често не може да ги направи естествена част от собствения си сайт.

Третият проблем е липсата на достатъчно гъвкава персонализация. Настройки само на цветове или свободен CSS не са достатъчни за последователно брендиране при различни компоненти и различни сайтове.

Четвъртият проблем е трудната поддръжка на интеграции с различни доставчици. Ако всеки доставчик изисква отделна логика, системата става трудна за развитие. По-подходящ е общ механизъм за конектори, при който различните източници се свеждат до единен вътрешен модел.

Петият проблем е смесването на съдържание, изглед и публикуване. Една и съща група оферти може да бъде показана по различен начин в различни сайтове. Затова тези понятия трябва да бъдат разделени в модела на платформата.

2.5 Обосновка на избрания подход

За Odisea е избран подход с вградими уеб компоненти и програмен интерфейс, защото той адресира ограниченията на ръчното съдържание и `iframe` интеграциите. Офертите могат да се управляват централизирано, а визуализацията им да се извършва директно в сайта на агенцията.

Компонентният подход позволява всяка агенция да използва готови елементи за представяне на оферти, без да изгражда цялата логика самостоятелно. Чрез система от теми и дизайн токени компонентите могат да бъдат съобразени с визуалната идентичност на конкретния сайт. Така се запазва баланс между повторна използваемост и персонализация.

В модела на платформата се разделят няколко основни понятия. Офертата описва туристическия продукт. Колекцията групира избрани

оферти. Изгледът определя начина на представяне. Темата управлява визуалните настройки. Публикацията свързва тези елементи в публично достъпен ресурс. Това разделение намалява дублирането и позволява една и съща информация да бъде използвана в различни контексти.

Подходът е подходящ и за бъдещо разширяване. След реализиране на основния механизъм за публикуване могат да бъдат добавени конектори към доставчици, различни канали за разпространение, аналитика и допълнителни функции за запитвания или резервации. Следващата глава конкретизира функционалните и нефункционалните изисквания, които произтичат от този анализ.

Глава 3: Изисквания към системата

В тази глава са описани основните изисквания към платформата Odisea. Те определят какви действия трябва да поддържа системата, какви качествени характеристики трябва да притежава и какви роли участват в работата с нея. Целта е да се постави ясна основа за архитектурата и техническата реализация, които са разгледани в следващите глави.

3.1 Функционални изисквания

Функционалните изисквания описват основните възможности на системата. Те обхващат управлението на оферти, създаването на колекции и публикации, вграждането в сайтове на агенции, обработката на запитвания и интеграциите с външни доставчици. Изискванията са обобщени в таблица 3.1.

Таблица 3.1. Основни функционални изисквания към платформата

Код	Изискване	Основна роля
ФИ-1	Регистрация на нов потребител със създаване на профил на агенция или туроператор	Анонимен потребител
ФИ-2	Вход, изход и поддържане на потребителска сесия	Всички регистрирани потребители
ФИ-3	Управление на оферти чрез създаване, редактиране, публикуване и оттегляне	OperatorAdmin
ФИ-4	Групово въвеждане на оферти от текстов или структуриран формат	OperatorAdmin
ФИ-5	Управление на връзки към външни доставчици	OperatorAdmin
ФИ-6	Ръчно стартиране на внос и преглед на резултата от изпълнението	OperatorAdmin
ФИ-7	Периодична синхронизация на оферти от външни източници	Система
ФИ-8	Създаване и редактиране на колекции от оферти	AgencyAdmin, AgencyEditor
ФИ-9	Настройване на теми чрез дизайн токени	AgencyAdmin, AgencyEditor
ФИ-10	Създаване на изгледи за визуализиране на оферти	AgencyAdmin, AgencyEditor
ФИ-11	Създаване на публикация от колекция, изглед и тема	AgencyAdmin

ФИ-12	Генериране на публичен ключ и код за вграждане	AgencyAdmin
ФИ-13	Зареждане на публикуваните оферти чрез вградими компоненти	Турист
ФИ-14	Ограничаване на достъпа чрез разрешени домейни и ключове	AgencyAdmin
ФИ-15	Приемане и преглед на запитвания от крайни потребители	AgencyAdmin, AgencyEditor
ФИ-16	Изпращане на известия към външни системи при настъпване на събитие	AgencyAdmin
ФИ-17	Управление на търговски достъп между туроператори и агенции	OperatorAdmin, AgencyAdmin
ФИ-18	Събиране и преглед на обобщена аналитична информация	AgencyAdmin, AgencyEditor

Изискванията показват, че платформата не се ограничава до визуализиране на туристически оферти. Тя трябва да поддържа пълния процес по тяхното въвеждане, подбиране, публикуване и проследяване. Това налага ясно разграничение между действията на туроператорите, туристическите агенции, крайните потребители и автоматизираните системни процеси.

3.2 Нефункционални изисквания

Нефункционалните изисквания определят качеството на работа на системата. Те са особено важни, защото Odisea обслужва повече от една организация и трябва да гарантира сигурност, предвидимост и лесна поддръжка.

3.2.1 Сигурност

Системата трябва да осигурява защитен достъп до административните функции. Всеки потребител трябва да има права, съобразени с ролята си и организацията, към която принадлежи. Данните на различните наематели трябва да бъдат изолирани, така че потребител от една организация да няма достъп до ресурси на друга.

Паролите не трябва да се съхраняват в открит вид. Публичните входни точки, използвани от вградените компоненти, трябва да бъдат защитени чрез ключове за достъп и списък с разрешени домейни. Системата трябва да ограничава прекомерен брой заявки към публичните крайни точки и да връща грешки без разкриване на вътрешни технически детайли.

3.2.2 Производителност

Публичното зареждане на оферти трябва да бъде достатъчно бързо, за да не влошава потребителското изживяване в сайта на агенцията. Данните, които се използват от вградените компоненти, трябва да могат да се кешират, когато това не нарушава актуалността на съдържанието.

Операции като внос от външни доставчици и синхронизация на голям брой оферти не трябва да блокират основната работа на системата. При по-бавна връзка към доставчик останалите функции на платформата трябва да продължат да работят нормално. В рамките на настоящата версия изискванията за производителност са определени качествено, а подробното натоварващо тестване се разглежда като следваща стъпка.

3.2.3 Надеждност

Системата трябва да обработва частични грешки без прекъсване на целия процес. Ако част от офертите при внос са невалидни, валидните записи трябва да бъдат обработени, а проблемните да бъдат отчетени. Ако известие към външна система не бъде доставено, това не трябва да води до загуба на основните данни.

Всеки процес по внос или синхронизация трябва да оставя запис за своето изпълнение. Така администраторите могат да проследят кога е стартиран процесът, какъв е резултатът и какви грешки са възникнали.

3.2.4 Разширяемост и поддръжка

Платформата трябва да позволява добавяне на нови доставчици, нови типове изгледи и нови визуални теми без сериозни промени в основната логика. За тази цел отделните части на системата трябва да имат ясно определени отговорности.

Конекторите към външни доставчици трябва да следват общ модел на работа. Темите трябва да използват повторно използваеми дизайн токени, а вградените компоненти трябва да могат да бъдат развивани независимо от конкретен сайт на агенция. В настоящата версия не се поставя изискване за отделни микросървиси, тъй като това би увеличило сложността на разработката и внедряването.

3.2.5 Съвместимост

Вградените компоненти трябва да работят в съвременни браузъри и да не зависят от конкретна клиентска рамка в сайта на агенцията. Сървърният интерфейс трябва да предоставя данни във формат, който може да бъде използван от различни клиенти и външни системи.

Системата трябва да може да се стартира в контролирана среда за разработка и внедряване. Това улеснява повторяемостта на настройките и намалява риска от различно поведение между локална и продукционна среда.

3.2.6 Удобство за потребителя

Административният интерфейс трябва да бъде разбираем за потребители без задълбочени технически познания. Основните действия,

като създаване на колекция, настройване на тема и публикуване на оферти, трябва да бъдат организирани в логична последователност.

Платформата трябва да предоставя ясна обратна връзка при успешно действие, грешка или непълни данни. Публичните компоненти трябва да се вписват визуално в сайта на агенцията и да не създават усещане за отделна външна система.

3.3 Потребителски роли

Системата дефинира няколко роли, чрез които се ограничава достъпът до различните функции. Разпределението е представено в таблица 3.2.

Таблица 3.2. Потребителски роли и основни права

Роля	Описание	Основни права
PlatformAdmin	Администратор на платформата	Управление на системни настройки и достъп до всички организации
OperatorAdmin	Администратор на туроператор	Управление на оферти, връзки към доставчици и достъп на агенции
AgencyAdmin	Администратор на агенция	Управление на колекции, теми, изгледи, публикации, домейни и известия
AgencyEditor	Редактор в агенция	Редактиране на колекции, теми и изгледи, както и преглед на запитвания
Турист	Краен потребител	Разглеждане на публикувани оферти и изпращане на запитване

Ролите са съобразени с реалното разделение между участниците в туристическия процес. Туроператорът управлява съдържанието, агенцията управлява начина на представяне, а туристът използва публикуваната информация през сайта на агенцията. Администраторът на платформата има отделна роля, предназначена за общо управление и поддръжка.

3.4 Основни потребителски сценарии

Основните сценарии описват как различните роли използват системата в типични ситуации. Те служат като основа за по-детайлното проектиране на процесите в следващата глава.

В сценария на туроператора потребителят създава профил на оператор и започва да въвежда оферти. Това може да стане ръчно, групово или чрез връзка към външен доставчик. След проверка офертите

се публикуват и стават достъпни за агенциите, които имат право да ги използват.

В сценария на агенцията потребителят създава профил на агенция, избира оферти от наличния каталог и ги подрежда в колекция. След това настройва изглед и тема, създава публикация и получава код за вграждане. Кодът се поставя в сайта на агенцията, където офертите се визуализират за крайните потребители.

В сценария на туриста крайният потребител разглежда оферти в сайта на агенцията. Компонентите трябва да изглеждат като естествена част от страницата. При интерес туристът изпраща запитване, което постъпва при съответната агенция.

В сценария на системната синхронизация платформата периодично проверява външни източници за промени. Новите или обновени оферти се обработват, а проблемите се записват за последващ преглед от администратор.

3.5 Ограничения и допускания

Настоящата версия на проекта е разработена при няколко ограничения. Пълният резервационен процес, включително плащане, издаване на документи и окончателно потвърждение на резервация, не е част от текущия обхват. Основният фокус е върху публикуването, вграждането и получаването на запитвания.

Приема се, че туроператорите могат да предоставят оферти в подходящ за обработка формат. Приема се също, че агенциите имат достъп до собствените си сайтове и могат да поставят предоставения код за вграждане. Крайните потребители се разглеждат като анонимни посетители, които използват съвременен браузър.

Някои функционалности са предвидени като основа за бъдещо развитие. Такива са по-задълбочената автоматизация на доставчиците, външната автентикация, разширената аналитика и пълният резервационен поток. Следващата глава разглежда архитектурата, чрез която описаните изисквания могат да бъдат реализирани.

Глава 4: Архитектура и проектни решения

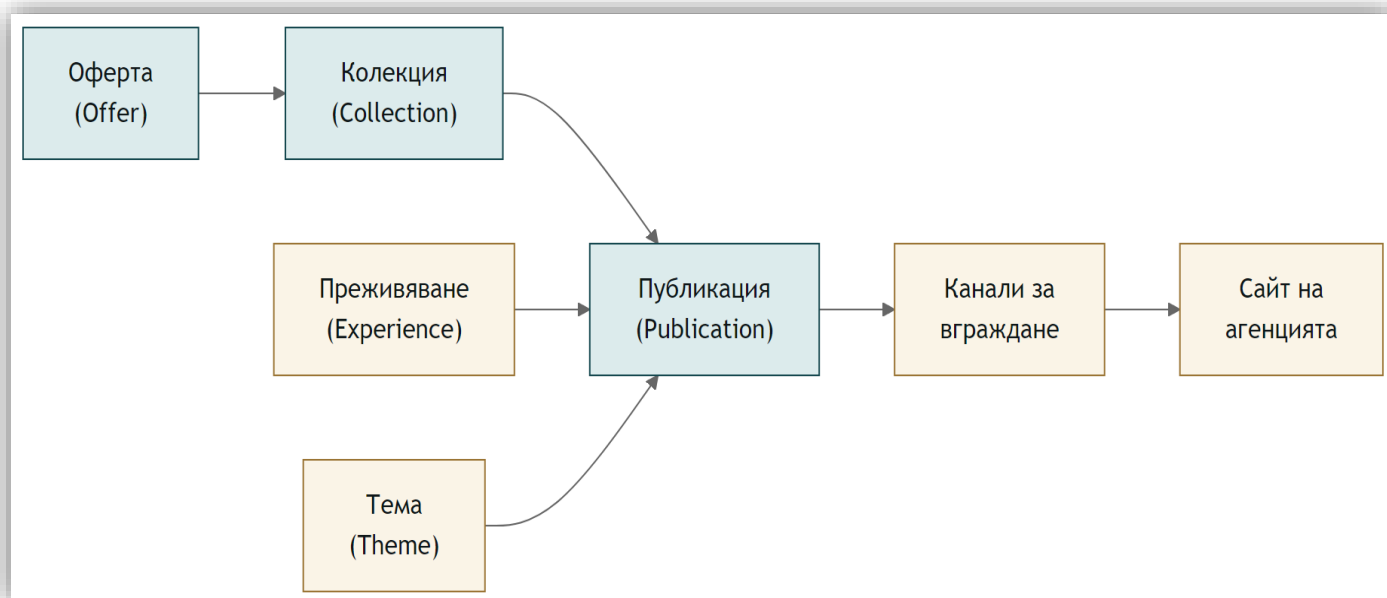
Настоящата глава описва архитектурата на платформата Odisea и основните проектни решения, чрез които изискванията от предходната глава се превръщат в цялостен системен модел. Акцентът е поставен върху разделянето на отговорностите, комуникацията между основните части, модела на данните и сигурността в среда с множество организации.

За представяне на архитектурата се използва моделът C4, тъй като той позволява системата да бъде разгледана на няколко нива на детайлност. Контекстната диаграма показва участниците и външните системи, контейнерната диаграма показва основните изпълними части, а компонентните диаграми разглеждат вътрешната организация на ключови модули. Допълнително е използван и моделът 4+1, за да се опише архитектурата от различни гледни точки.

4.1 Обща архитектурна концепция

Основната архитектурна идея на Odisea е разделяне на съдържанието, подбора, визуалното представяне и публичното разпространение. Това разделение позволява една и съща туристическа оферта да бъде използвана в различни контексти, без да се дублира и без да се обвързва с конкретен сайт или конкретен визуален изглед.

В основата на модела стои офертата. Тя описва туристическия продукт чрез нормализирани данни като дестинация, период, цена, транспорт, хотел, описание и допълнителни условия. Колекцията определя кои оферти да бъдат включени в дадена селекция. Изгледът описва начина на представяне, например списък, решетка, търсачка или въртележка. Темата съдържа визуалните настройки, чрез които компонентите се съобразяват с облика на сайта на агенцията. Публикацията обединява колекция, изглед и тема в публично достъпен ресурс.



Фигура 4.1. Концептуален модел на платформата

Това разделение има практическо значение. Ако дадена оферта бъде обновена в каталога, промяната може да достигне до всички публикации, които я използват. Ако агенцията иска да промени визуалния стил, тя може да редактира темата, без да променя офертите. Ако е необходим нов начин на представяне, може да бъде създаден нов изглед върху съществуваща колекция. Така системата постига повторна използваемост и намалява риска от несъответствия между съдържание и визуализация.

4.2 Избрана архитектурна методология

Сървърната част на платформата следва принципите на чистата архитектура. При този подход бизнес логиката се поставя в центъра на системата, а техническите детайли като база данни, уеб интерфейс и външни интеграции остават във външните слоеве [3]. Основното правило е зависимостите да сочат навътре, към ядрото на приложението.

Архитектурата може да бъде разгледана чрез четири основни слоя.

Първият слой е слойът на предметната област. Той съдържа основните бизнес понятия, правила и връзки между тях. В него попадат същности като оферта, колекция, тема, изглед, публикация, организация и запитване. Този слой не трябва да зависи от конкретна база данни, уеб технология или външна услуга.

Вторият слой е приложният слой. Той описва използваните случаи на системата, например създаване на публикация, обновяване на оферта, приемане на запитване или стартиране на внос от доставчик. Тук се

координират действията между домейн модела и необходимите външни зависимости, но без да се обвързва логиката с конкретна инфраструктура.

Третият слой е инфраструктурният слой. В него се намират конкретните реализации за достъп до база данни, работа с файлове, комуникация с външни доставчици, изпращане на известия и други технически операции.

Четвъртият слой е входният слой на системата. Той включва крайните точки на приложния интерфейс, публичните входни точки за вградените компоненти и административните операции. Този слой приема заявки, валидира входните данни и предава управлението към приложния слой.

Тази структура е избрана, защото позволява развитие на платформата без силно обвързване между отделните части. Например промяна в начина на съхранение на данните не трябва да налага промяна в бизнес правилата. По същия начин добавянето на нов тип конектор не трябва да променя логиката за работа с вече нормализирани оферти.

4.3 Контекстна диаграма по C4

Моделът C4 започва с контекстна диаграма, която показва системата като едно цяло и описва връзките ѝ с хората и външните системи [5]. При Odisea основните участници са туроператорът, туристическата агенция, администраторът на платформата и крайният турист.

Туроператорът предоставя съдържание под формата на оферти. Агенцията подбира тези оферти, настройва начина им на представяне и ги публикува в своя сайт. Администраторът на платформата управлява системни настройки и подпомага поддръжката. Туристът не работи директно с административната част на Odisea, а вижда публикуваните оферти през сайта на агенцията.

Външните системи включват източници на оферти, сайтове на агенции и системи, към които могат да се изпращат известия при събития. Такива събития могат да бъдат получено запитване, промяна в публикация или друга важна бизнес операция.



Фигура 4.2. Контекстна диаграма по модела C4

Контекстната диаграма е важна, защото ясно отделя платформата от външната среда. Тя показва, че Odisea не е просто административен панел за оферти, а посредническа платформа между доставчици на съдържание, агенции и крайни потребители.

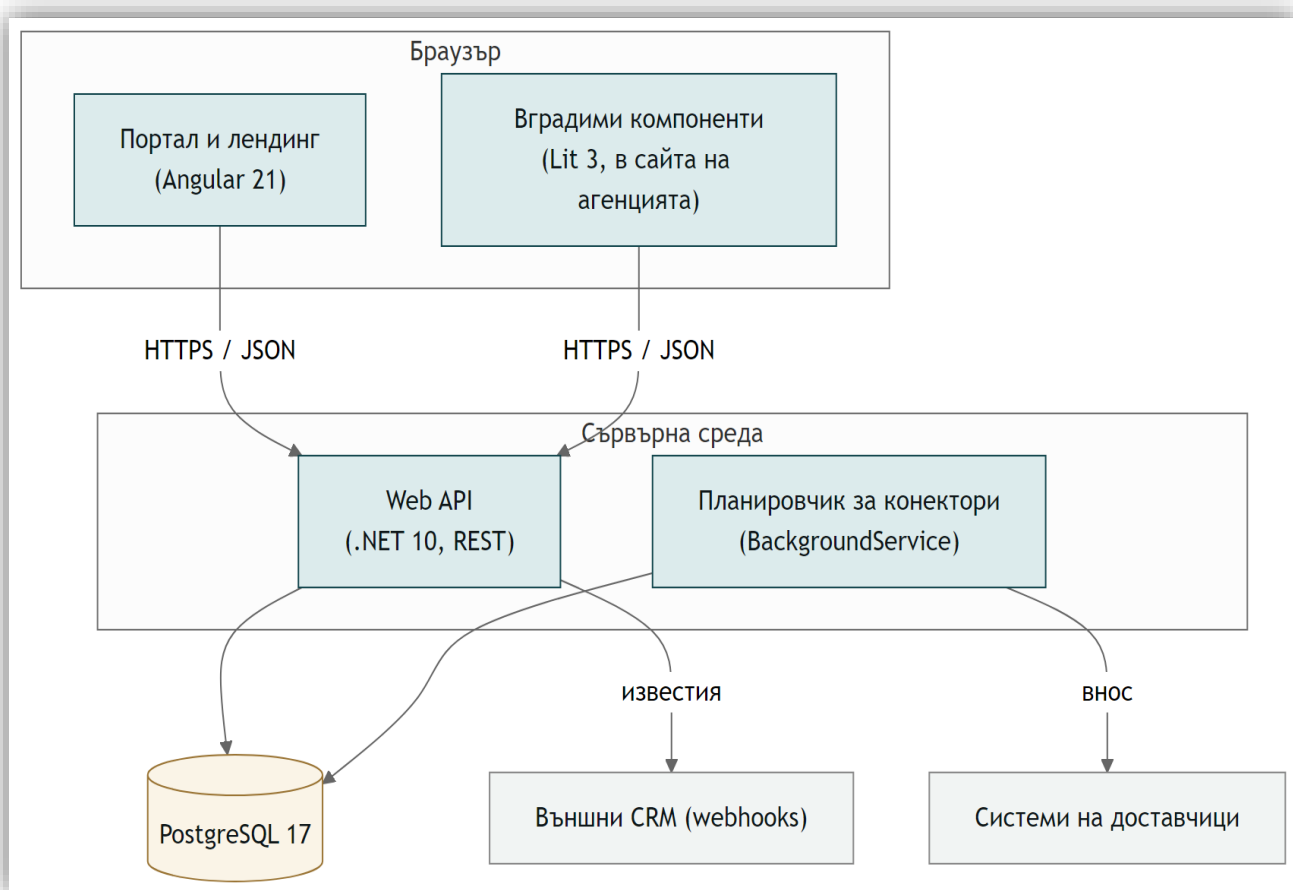
4.4 Контейнерна диаграма по C4

Второто ниво на C4 представя основните изпълними части на системата. При Odisea те могат да бъдат разгледани като административен портал, публичен сайт, пакет с вградими компоненти, сървърен приложен интерфейс, фоновата услуга за синхронизация и база данни.

Административният портал се използва от туроператори, агенции и администратори. Чрез него се управляват оферти, колекции, теми, изгледи, публикации, запитвания и настройки. Публичният сайт представя основна информация за платформата и осигурява вход към административната част.

Вградимите компоненти се изпълняват в сайтовете на агенциите. Те зареждат публична конфигурация на публикацията, получават необходимите оферти и ги визуализират според зададената тема и изглед. Тяхната задача е да предоставят съдържание, което изглежда като естествена част от сайта на агенцията.

Сървърният приложен интерфейс обслужва административните операции, публичното зареждане на публикации и приемането на запитвания. Фоновата услуга изпълнява задачи, които не трябва да блокират основната работа на потребителите, например синхронизация с външни доставчици. Базата данни съхранява потребители, организации, оферти, публикации, теми, събития и свързаните с тях настройки.



Фигура 4.3. Контейнерна диаграма по модела C4

Контейнерният изглед показва, че системата е организирана като модулен монолит с ясно разделени отговорности. Това позволява лесна разработка и внедряване в началния етап, като същевременно запазва възможност за бъдещо отделяне на части при реална необходимост.

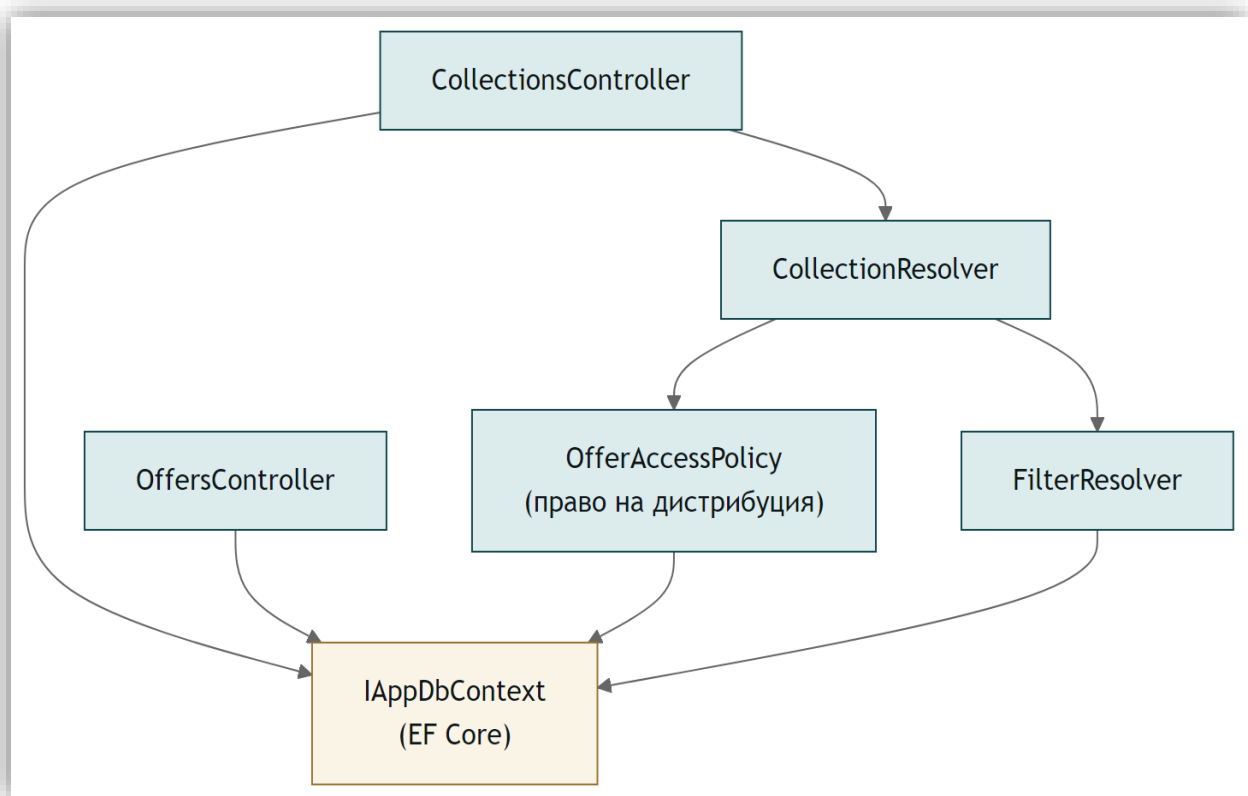
4.5 Компонентни диаграми по C4

Третото ниво на C4 разглежда вътрешната структура на отделни контейнери. За Odisea най-съществени са компонентите, свързани с каталога, публикациите и конекторите към външни доставчици.

4.5.1 Компоненти на каталога и публикациите

Каталогът съдържа нормализираните туристически оферти. Върху него стъпват колекциите, които определят кои оферти да бъдат показани. Колекциите могат да използват филтри, ръчно избрани оферти и изключени записи. След това изгледът и темата определят как тези оферти ще бъдат представени.

Основните компоненти в тази част са крайните точки за управление, приложните услуги за колекции и публикации, домейн правилата за валидност и достъп, както и абстракциите за достъп до данните. Крайните точки не трябва да съдържат бизнес логика. Те приемат заявката и я предават към приложния слой, където се изпълняват правилата.



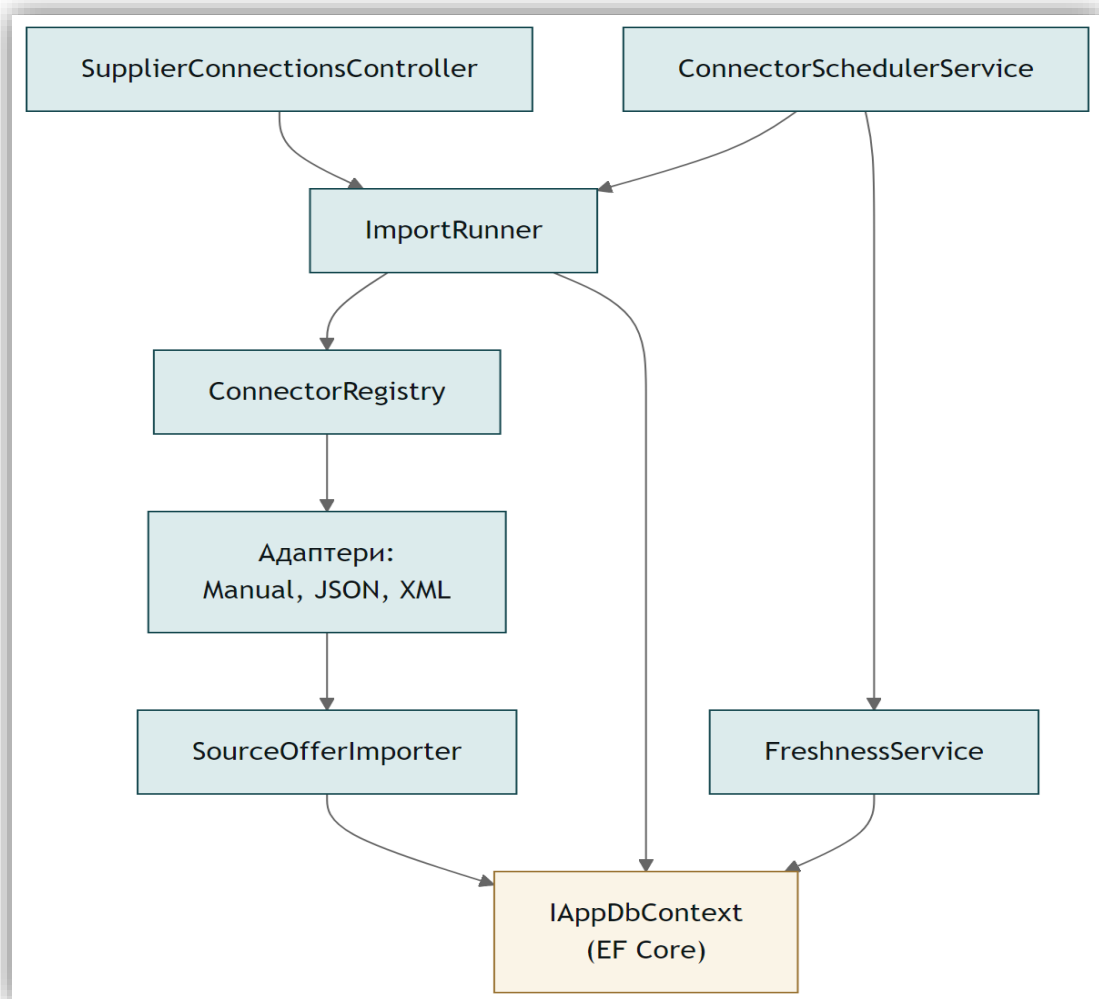
Фигура 4.4. Компонентна диаграма на каталога и филтрирането

Тази структура улеснява поддръжката, защото правилата за публикуване и подбор на оферти са отделени от начина, по който потребителят достъпва системата. Така една и съща логика може да бъде използвана както от административния портал, така и от други бъдещи клиенти.

4.5.2 Компоненти на механизма за конектори

Механизмът за конектори отговаря за получаването на оферти от външни източници. Той трябва да поддържа различни формати и протоколи, без всяка интеграция да изисква промяна в основната логика на системата.

Процесът започва от планировчик или ръчно стартирана операция. След това изпълнителят на внос избира подходящ адаптер според типа на връзката. Адаптерът извлича данните и ги преобразува до общ междинен формат. След това общият импортер валидира информацията, нормализира я и обновява каталога. Ако даден запис е невалиден, той се отчита като грешка, без да спира обработката на останалите валидни записи.



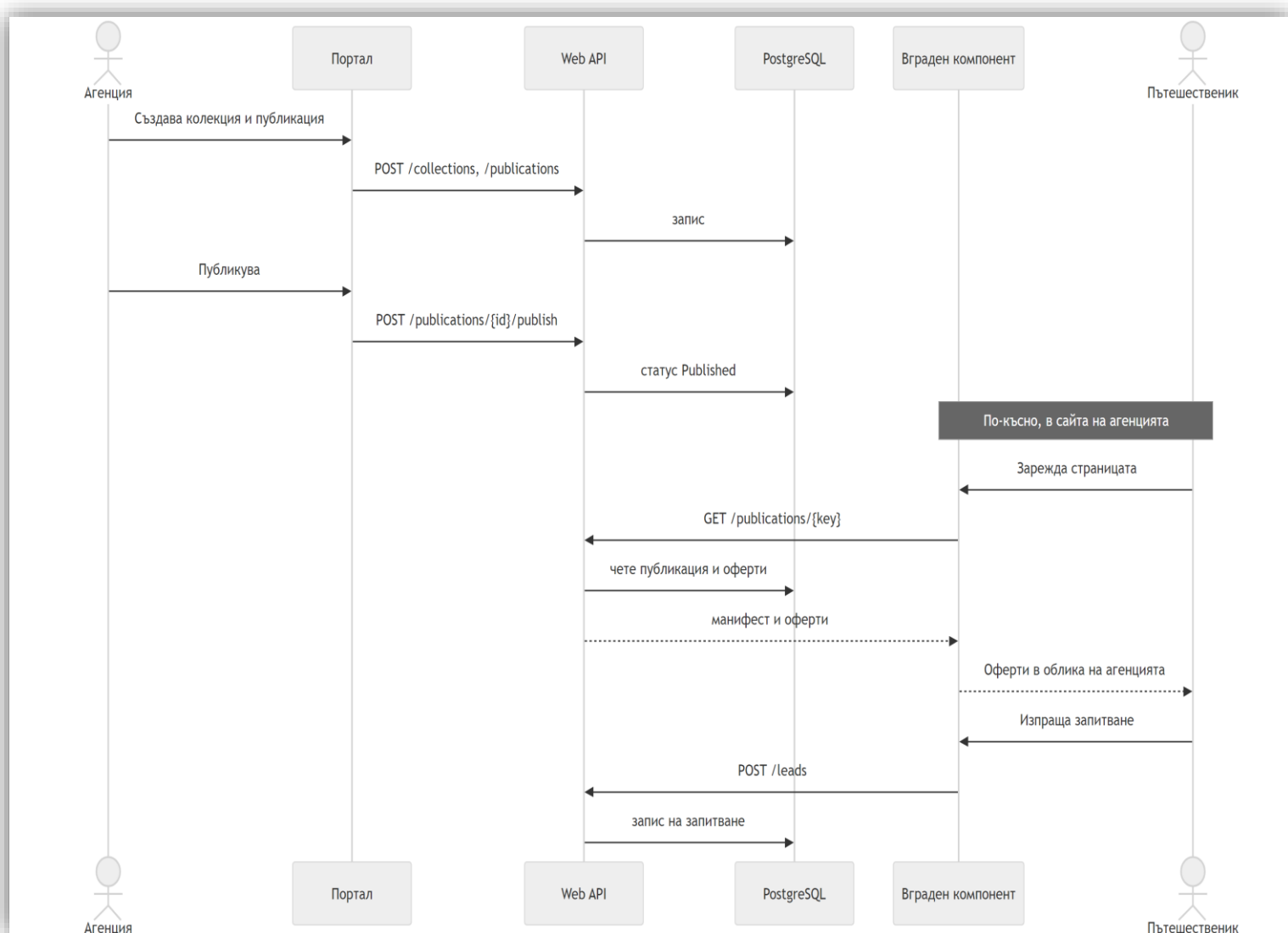
Фигура 4.5. Компонентна диаграма на механизма за конектори

Този подход позволява добавяне на нови доставчици чрез нови адаптери, без промяна в останалите части на системата. Така интеграциите се превръщат в разширяема способност на платформата, а не в поредица от отделни технически решения.

4.6 Комуникация между компонентите

Основният комуникационен поток започва в административния портал. Агенцията създава колекция от оферти, избира изглед, настройва тема и създава публикация. След това системата генерира публичен ключ и код за вграждане. Този код се поставя в сайта на агенцията.

Когато посетител отвори страницата на агенцията, вграденият компонент използва публичния ключ, за да зареди манифеста на публикацията. Манифестът съдържа информация за използвания изглед, тема и необходимите публични настройки. След това компонентът зарежда офертите и ги визуализира в страницата. Ако туристът изпрати запитване, то се записва към съответната агенция и може да бъде прегледано в административния портал.



Фигура 4.6. Диаграма на последователността за публикуване и вграждане

Този поток показва защо публикацията е отделно понятие в модела. Тя служи като стабилна публична точка за достъп, без да разкрива

вътрешната структура на колекции, теми и изгледи. По този начин външният сайт работи с ясен публичен ресурс, а вътрешното управление остава контролирано от платформата.

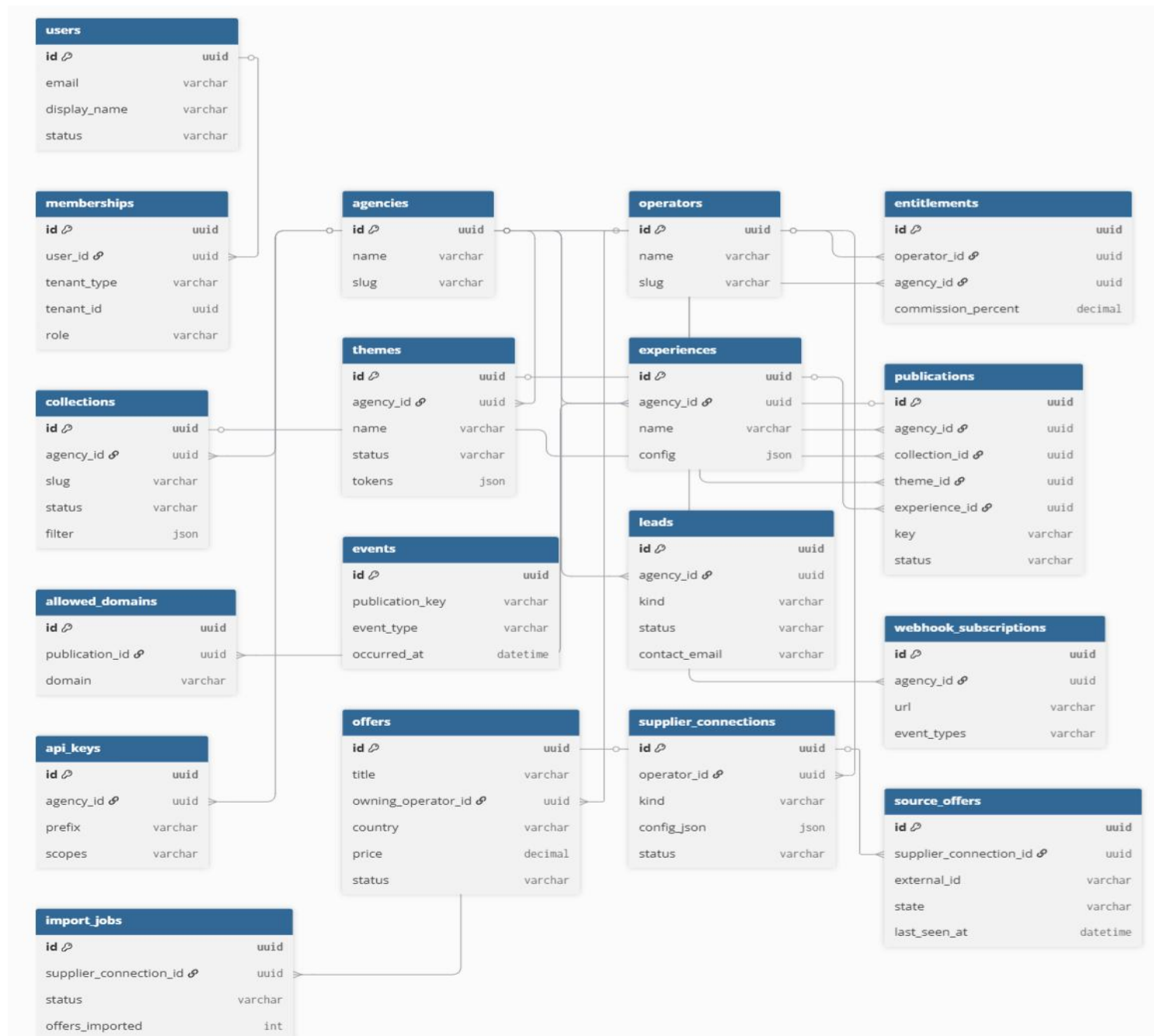
4.7 Модел на данните и база данни

Моделът на данните отразява основните понятия в платформата. Централно място заема организацията, която представлява работна единица в системата. Тя може да бъде туристическа агенция, туроператор или друг тип участник според бъдещо разширение на платформата. Потребителите се свързват с организации чрез членства, които определят ролята им.

Туроператорът управлява оферти и връзки към доставчици. Връзките към доставчици съдържат настройки за източника, а всяко изпълнение на внос оставя запис за резултата си. Така може да се проследи произходът и актуалността на внесената информация.

Агенцията управлява колекции, теми, изгледи и публикации. Колекцията определя избора на оферти. Темата съдържа визуалните настройки. Изгледът описва начина на представяне. Публикацията свързва тези три части и предоставя публичен ключ. Към публикацията могат да бъдат добавени разрешени домейни, аналитични събития и запитвания от крайни потребители.

Фигура 4.7. Модел на данните (ER диаграма)



Няколко решения в модела са важни за гъвкавостта на системата. Първо, съдържанието и публикуването са разделени. Това позволява една оферта да участва в много публикации. Второ, визуалните настройки са отделени в тема, което позволява повторно използване на един и същ стил. Трето, връзката към източника на офертата се запазва, което осигурява проследимост при обновяване или отстраняване на грешки.

4.8 Сигурност и управление на достъпа

Сигурността в Odisea е свързана основно с три въпроса: кой потребител извършва действието, към коя организация принадлежи и дали има право да достъпва конкретния ресурс. Тъй като системата обслужва множество организации, проверката на организационния контекст е също толкова важна, колкото и проверката на ролята.

Достъпът до административните функции се управлява чрез автентикация и ролеви права. Потребителят може да изпълнява само действията, които са позволени за неговата роля в рамките на конкретната организация. Така администратор на една агенция не може да управлява публикации на друга агенция, а администратор на туроператор не може да променя ресурси, които не принадлежат на неговата организация.

Публичните входни точки имат различен модел на защита. Вградените компоненти не работят с потребителска сесия, а с публичен ключ на публикацията. За да се намали рискът от неправомерно използване, публикацията може да бъде ограничена до списък с разрешени домейни. Така компонентът може да бъде зареждан само от сайтове, които са предварително одобрени от агенцията.

Изходящите известия към външни системи трябва да позволяват проверка на произхода. Затова архитектурата предвижда подписване на съобщенията. Приемането на публични запитвания трябва да бъде защитено и от прекомерен брой заявки, тъй като тази част е достъпна за анонимни потребители.

Грешките се връщат в стандартизиран вид. Те трябва да бъдат достатъчно ясни за потребителя или разработчика, но без да разкриват вътрешни детайли за инфраструктурата, заявките към базата или вътрешната организация на кода.

4.9 Обосновка на архитектурните решения

Изборът на модулен монолит е съобразен с текущия обхват на проекта. Платформата има ясно разграничени области, но все още няма доказана необходимост всяка от тях да бъде отделна услуга. Микросървисният подход би добавил сложност при внедряване,

наблюдение, комуникация и поддръжка. Модулният монолит позволява по-бърза разработка, по-лесно тестване и по-ниска оперативна сложност.

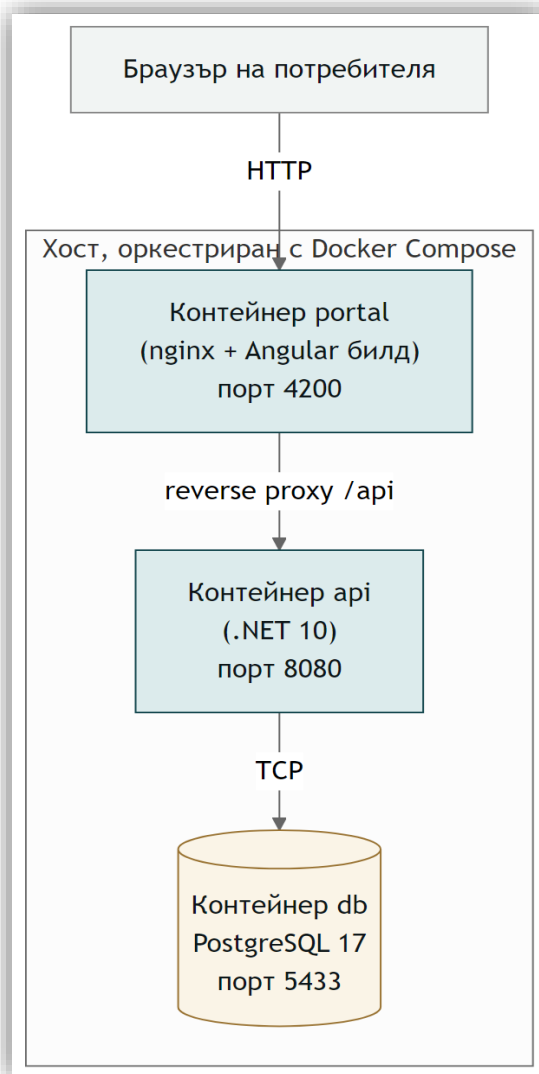
В същото време вътрешните граници между модулите са запазени. Това означава, че при бъдещо нарастване на натоварването или екипа отделни части могат да бъдат изведени самостоятелно. Например механизмът за конектори или публичното обслужване на публикации могат да бъдат отделени, ако реалната употреба го наложи.

Изборът на вградими компоненти е пряко свързан с продуктовия проблем. За разлика от `iframe`, компонентите могат да бъдат част от страницата на агенцията и да приемат нейния визуален стил. Това позволява по-добро потребителско изживяване и по-добра интеграция със сайта.

Тематизирането чрез дизайн токени е избрано вместо свободно стилизиране, защото осигурява по-предвидим резултат. Агенцията може да управлява основните визуални характеристики, без да нарушава вътрешната структура на компонентите. Това намалява риска от счупен интерфейс и улеснява повторното използване на теми.

Механизмът за конектори е проектиран с общ договор и заменяеми адаптери. Това решение намалява зависимостта от конкретен доставчик и позволява постепенно добавяне на нови източници. Всяка интеграция трябва да довежда данните до общ вътрешен модел, така че останалата част от системата да работи с еднаква структура.

Физическото разполагане на контейнерите е показано на фигура 4.8.



Фигура 4.8. Диаграма на разполагането (deployment)

В обобщение, архитектурата на Odisea търси баланс между гъвкавост и контрол. Тя избягва ненужната сложност на ранния етап, но запазва ясни граници между основните области. Това позволява платформата да бъде развивана постепенно, без да се нарушава основният модел на работа. Следващата глава разглежда конкретните технологии и среди, чрез които тази архитектура е реализирана.

Глава 5: Технологии и среди за разработка

Тази глава представя технологиите, използвани при реализацията на платформата Odisea. Фокусът е върху ролята на всяка технология в системата и причините за нейния избор. Подробностите за структурата на кода и конкретната реализация са разгледани в следващата глава.

5.1 Преглед на използваните технологии

Платформата използва няколко основни технологии, всяка от които изпълнява ясно определена задача. Angular се използва за административния портал и публичния сайт. Lit се използва за вградимите компоненти, които трябва да работят в сайтове на различни агенции. Сървърната част е реализирана с .NET и ASP.NET Core, а данните се съхраняват в PostgreSQL.

За локално стартиране и изолиране на средата се използва Docker. Качеството на разработката се подпомага чрез автоматизирани тестове и непрекъснатата интеграция. Така отделните части на системата могат да се разработват и проверяват по предвидим начин.

5.2 Клиентска част

Административният портал и публичният сайт са реализирани с Angular [10]. Тази технология е подходяща за по-големи клиентски приложения, тъй като предоставя ясна структура за компоненти, маршрутизация, форми, HTTP комуникация и защита на достъпа. Това позволява порталът да се разширява с нови екрани и функционалности, без да се нарушава цялостната организация на приложението.

В портала се изпълняват основните административни действия: управление на оферти, колекции, теми, изгледи, публикации, домейни, запитвания и настройки. Поради това е важно клиентската част да поддържа ясно разделение между страници, услуги и модели на данни.

Вградимите компоненти са реализирани с Lit [11]. Те се компилират до стандартни уеб компоненти и могат да бъдат използвани в сайтове, независимо от технологията, с която е изграден конкретният сайт. Това е важно, защото туристическите агенции могат да използват различни платформи и рамки.

Компонентите се настройват чрез входни параметри и CSS променливи. По този начин агенцията може да запази своя визуален стил, без да променя вътрешната логика на компонентите. Този подход е по-подходящ за вграждане в чужди сайтове от решение, което зависи от конкретна клиентска рамка.

5.3 Сървърна част

Сървърната част е реализирана с .NET и ASP.NET Core [7][8]. Тази среда предоставя стабилна основа за изграждане на REST интерфейс, автентикация, контрол на достъпа, валидиране на заявки и внедряване на зависимости. Тя е подходяща за система, която трябва да обслужва както административен портал, така и публични заявки от вградими компоненти.

Приложението е организирано по слоеве, в съответствие с архитектурата, описана в предходната глава. Сървърният интерфейс приема заявките и ги предава към приложния слой, където се изпълняват конкретните операции. Така крайните точки не съдържат основната бизнес логика, а служат като вход към нея.

Достъпът до данните се реализира чрез Entity Framework Core [9]. Той позволява работа с базата чрез модели, конфигурации и LINQ заявки. Това намалява нуждата от ръчно писане на SQL за стандартните операции и улеснява поддръжката на схемата чрез миграции.

Програмният интерфейс следва принципите на REST [15]. Ресурсите се адресират чрез ясни пътища, а обменът на данни се извършва във формат JSON. Документацията на интерфейса се генерира чрез OpenAPI и се преглежда чрез Swagger в среда за разработка. Грешките се връщат в стандартизиран формат, което улеснява обработката им от клиентските приложения [16].

5.4 База данни

За съхранение на данните се използва PostgreSQL [12]. Изборът е подходящ, защото платформата работи основно с релационни данни: потребители, организации, оферти, колекции, публикации, запитвания и права за достъп. Връзките между тези данни са важни и трябва да бъдат ясно дефинирани.

PostgreSQL предоставя и поддръжка на jsonb, което е полезно за по-гъвкави структури. Такива са например конфигурации на изгледи, филтри за колекции и настройки на теми. Така основните данни се пазят релационно, а променливите конфигурации могат да се съхраняват без създаване на прекалено сложна схема.

Промените по базата се управляват чрез миграции. Това позволява схемата да се развива заедно с приложението и улеснява възпроизвеждането на една и съща структура в различни среди.

5.5 Инфраструктура и DevOps подход

Docker се използва за стартиране на основните части на системата в контролирана среда [13]. Чрез контейнеризацията могат да бъдат описани сървърното приложение, базата данни и клиентската част с

техните зависимости. Това намалява разликите между локална среда и среда за внедряване.

Този подход улеснява и първоначалното стартиране на проекта. Вместо всяка зависимост да се настройва ръчно, средата може да бъде възстановена чрез предварително описана конфигурация. Това е особено полезно при разработка, тестване и бъдещо разгръщане.

Качеството на кода се подпомага чрез непрекъснатата интеграция. При промяна в хранилището могат да се изпълняват проверки за компилация, тестове и статичен анализ. Така проблеми в сървърната част, портала или библиотеката с компоненти могат да бъдат открити рано.

5.6 Външни услуги и интеграции

Платформата комуникира с външни системи в две основни посоки. Първата посока е към доставчиците на оферти. Данните могат да постъпват в различни формати, например XML или JSON, след което се преобразуват към вътрешния модел на системата.

Втората посока е към системи на агенциите. При определени събития платформата може да изпраща известия към външна система, например при получено запитване. Това позволява Odisea да се свързва с други инструменти, без да изисква от агенцията да следи само административния портал.

За удостоверяване на потребителите се използват маркери за достъп във формат JWT [14]. Публичните заявки от вградените компоненти използват отделен механизъм, основан на ключове за достъп и разрешени домейни. Така административният достъп и публичното зареждане на публикации са разделени.

5.7 Среда за разработка

Разработката на сървърната част се извършва основно във Visual Studio, тъй като средата предлага добра поддръжка за .NET проекти, дебъгване, управление на пакети и работа с тестове. За клиентската част се използва Visual Studio Code, който е удобен при работа с Angular, Lit, TypeScript и конфигурационни файлове.

Node.js се използва за изграждане и стартиране на клиентските проекти. Git се използва за управление на версиите, а чувствителните настройки се държат извън хранилището. Локалната инфраструктура се стартира чрез Docker, което улеснява работата с база данни и зависими услуги.

Тази среда е избрана, защото позволява бърза ежедневна разработка и сравнително лесно възпроизвеждане на проекта на друга машина.

5.8 Причини за избора на технологиите

Изборът на технологии се основава на три основни критерия: съответствие със задачата, стабилност и възможност за поддръжка.

Angular е подходящ за административния портал, защото предоставя цялостна структура за по-големи клиентски приложения. Lit е подходящ за вградимите компоненти, защото създава стандартни уеб компоненти, които не зависят от рамката на приемащия сайт. .NET и ASP.NET Core са подходящи за сървърната част, защото предоставят стабилна среда за програмен интерфейс, сигурност и работа със зависимости.

PostgreSQL е избран заради комбинацията между релационен модел и поддръжка на jsonb структури. Docker е избран, защото улеснява стартирането на системата и намалява разликите между средите. Entity Framework Core е естествен избор в .NET екосистемата, тъй като се интегрира добре с приложния модел и миграциите.

Така технологичният стек остава достатъчно широк, за да покрие нуждите на системата, но без излишно усложняване. Всяка технология има конкретна роля, а връзката между тях следва архитектурните решения, описани в предходната глава.

Глава 6: Техническа реализация

След като архитектурата и използваните технологии са уточнени, настоящата глава разглежда тяхното конкретно приложение в реализацията на платформата Odisea. Представени са структурата на проекта, основните части на клиентския и сървърния слой, реализацията на бизнес логиката, достъпът до базата данни, комуникацията между модулите, обработката на грешки и прилагането на мерките за сигурност.

Целта на главата не е да се показва целият изходен код, а да се илюстрират най-важните решения чрез кратки и представителни фрагменти. По този начин реализацията се описва на ниво, което показва логиката на системата, без да се натоварва изложението с излишни технически детайли.

6.1 Структура на проекта

Проектът е разделен на сървърна и клиентска част. Сървърната част следва слоевете, описани в архитектурната глава, а клиентската част съдържа административния портал, публичния сайт и библиотеката с вградими компоненти. Общата структура е показана по-долу.

```
odisea/  
  backend/  
    src/  
      Odisea.Domain/      # същности, стойностни обекти, изброими  
типове  
      Odisea.Application/  # използвани случаи, интерфейси, бизнес  
логика  
      Odisea.Infrastructure/ # EF Core, миграции, конектори, услуги  
      Odisea.WebAPI/       # контролери, middleware, конфигурация  
    tests/  
      Odisea.UnitTests/    # модулни и компонентни тестове  
  frontend/  
    portal/                # Angular портал и публичен сайт  
    components/            # Lit вградими компоненти  
  compose.yaml             # оркестрация на услугите
```

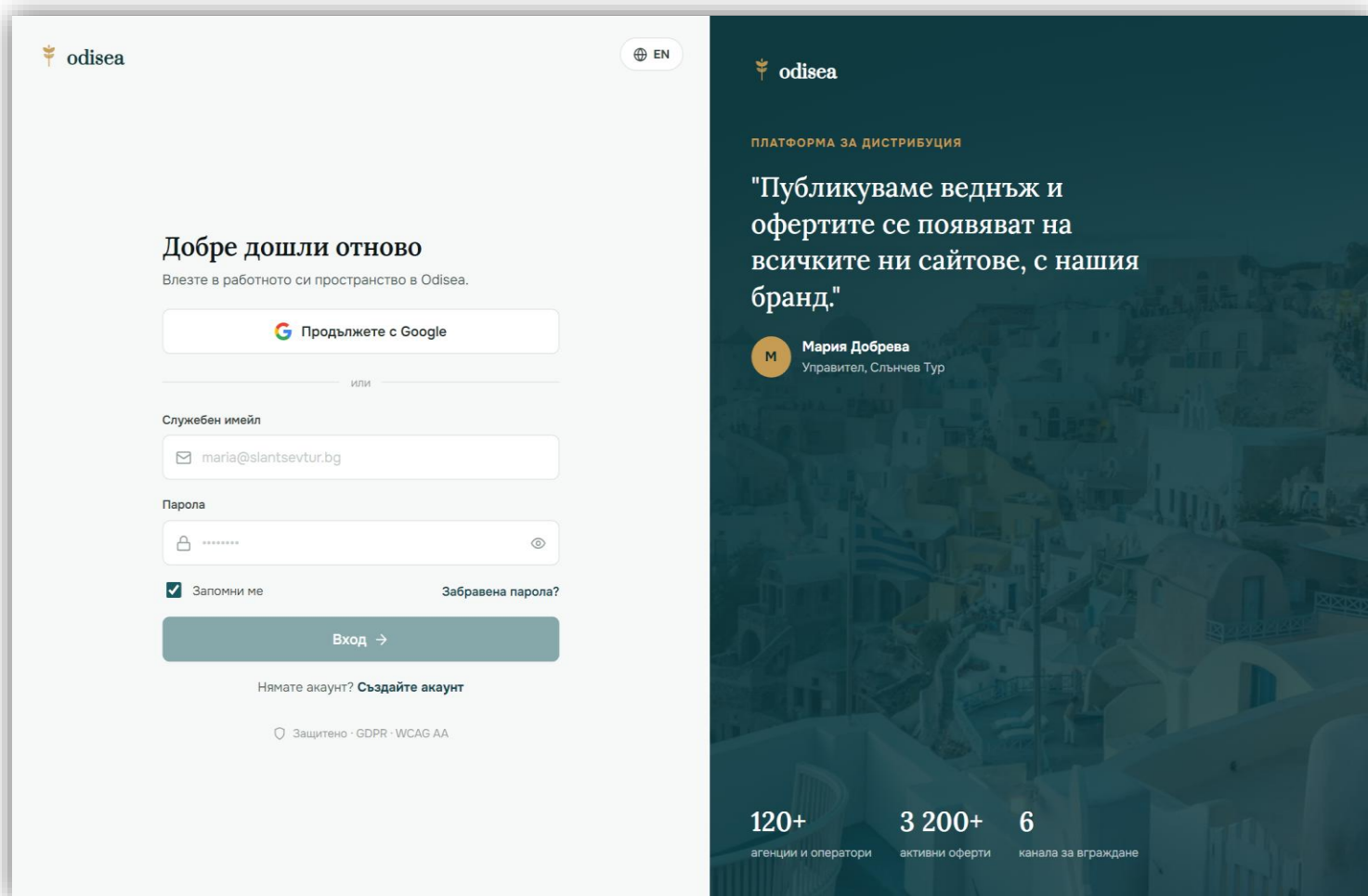
Тази структура поддържа ясно разделение между бизнес логика, инфраструктура и входни точки към системата. Слойът `Odisea.Domain` съдържа основните понятия и правила. `Odisea.Application` описва използваните случаи и работи с абстракции. `Odisea.Infrastructure` съдържа конкретните реализации за база данни, конектори и външни услуги. `Odisea.WebAPI` предоставя HTTP интерфейса и конфигурацията на приложението.

Клиентската част също е разделена според начина на използване. Порталът обслужва административните действия на агенции, туроператори и администратори. Библиотеката с компоненти е отделена, защото нейната цел е различна: тя трябва да работи в сайтове на агенции, независимо от технологията, с която са изградени.

6.2 Реализация на клиентския слой

Клиентският слой се състои от две основни части: административен портал и вградими компоненти. Порталът е реализиран с Angular и е организиран около основните работни екрани на системата. Всеки екран има собствен компонент, а комуникацията със сървъра е изнесена в отделни услуги.

Екранът за вход и регистрация е първата точка на взаимодействие с платформата. Той е реализиран като разделен изглед, при който от едната страна се намира формата, а от другата страна се представя марката и предназначението на системата.

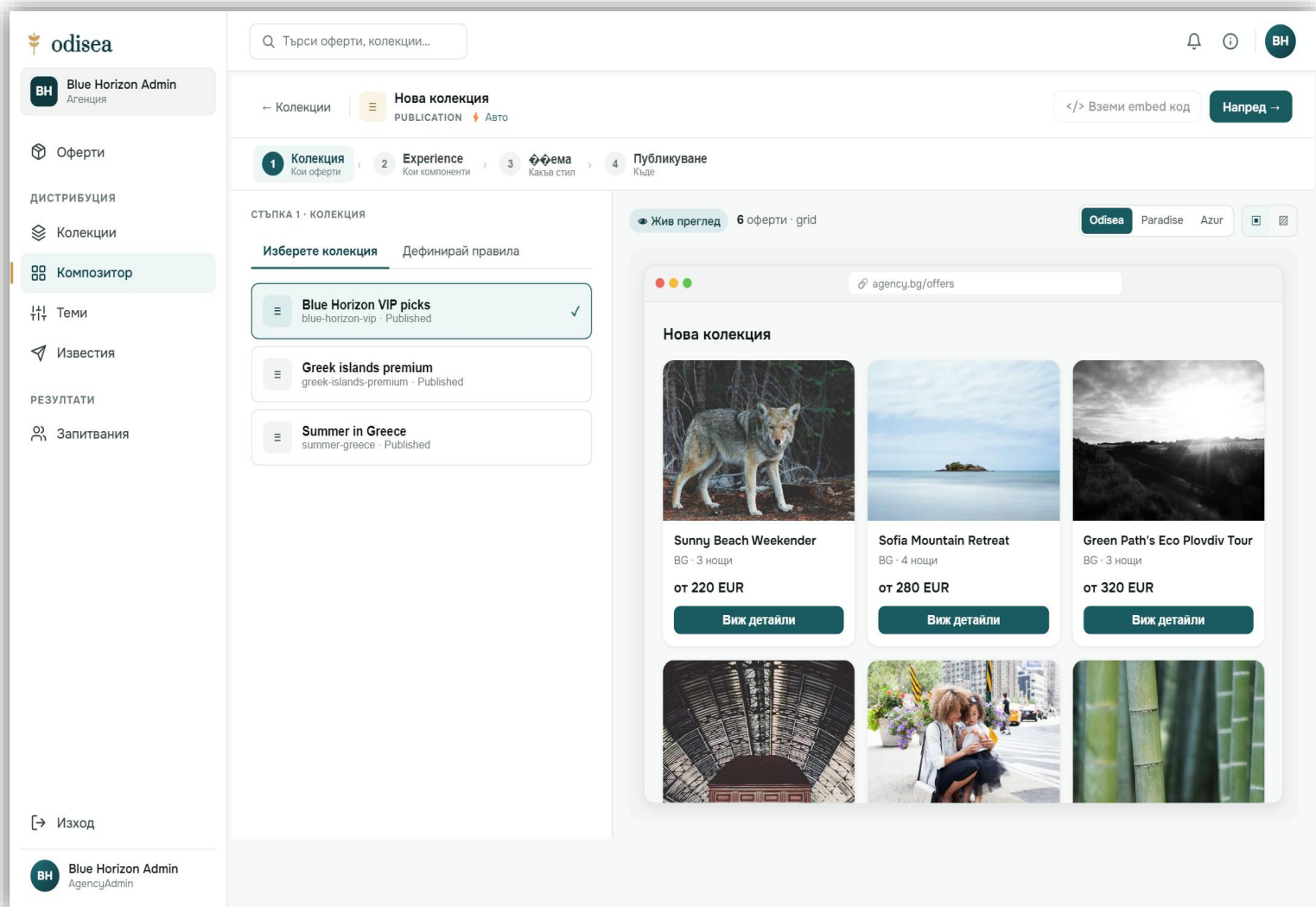


Фигура 6.1. Екран за вход в работното пространство

След вход в системата потребителят получава достъп до функционалности според своята роля и организация. Порталът използва клиентска логика за проверка на достъп до отделни маршрути, а сървърът извършва окончателната проверка при всяка заявка.

Най-същественият екран от гледна точка на агенцията е този за съставяне на публикации. В него потребителят избира колекция, изглед

и тема, след което получава преглед на крайния резултат. Това позволява проверка на публикацията преди тя да бъде използвана в реален сайт.



Фигура 6.2. Екран за съставяне на публикации с преглед в реално време

Вградимите компоненти са реализирани като отделна Lit библиотека. Те получават информация за публикацията чрез публичен ключ и визуализират офертите според конфигурацията, върната от сървъра. Компонентите използват CSS променливи, за да могат да бъдат стилизирани от приемащия сайт, без да се променя техният код.

Следният фрагмент показва пример за дефиниране на дизайн токени със стойности по подразбиране.

```
:host {
  --odc-accent: #1a5a61;
  --odc-surface: #ffffff;
  --odc-radius: 14px;
}

.card {
```

```

background: var(--odc-surface);
border-radius: var(--odc-radius);
}

.card__cta {
background: var(--odc-accent);
color: var(--odc-accent-ink);
}

```

Този подход позволява на агенцията да промени основни визуални характеристики като цвят, фон и радиус на елементите. В същото време вътрешната структура и поведението на компонента остават контролирани от платформата.

6.3 Реализация на сървърния слой

Сървърният слой предоставя HTTP интерфейс за административния портал, публичните компоненти и външните интеграции. Контролерите приемат заявки, извършват начална валидация и предават изпълнението към приложния слой. Те не съдържат основната бизнес логика, а служат като вход към нея.

Следният фрагмент показва примерен контролер за извличане на офертите на текущия оператор.

```

[ApiController]
[Route("api/v1/offers")]
public class OffersController(IAppDbContext db, IOperatorContext operatorCtx)
    : ControllerBase
{
    [Authorize(Policy = "OperatorAdmin")]
    [HttpGet("mine")]
    public async Task<IActionResult> Mine(CancellationToken ct)
    {
        var operatorId = operatorCtx.RequireOperator();

        var list = await db.Offers
            .Where(o => o.OwningOperatorId == operatorId)
            .OrderByDescending(o => o.CreatedAt)
            .ToListAsync(ct);

        return Ok(list.Select(o => o.ToDto()));
    }
}

```

В примера достъпът е ограничен чрез политика за роля, а данните се филтрират според текущия оператор. Така контролерът не връща всички оферти, а само тези, които принадлежат на организацията, от чието име работи потребителят.

Регистрацията на услугите е разделена по слоеве. В точката на стартиране се добавят само необходимите конфигурационни методи, а детайлите остават в съответните проекти. Това прави конфигурацията по-четима и намалява зависимостта между входния слой и вътрешните реализации.

6.4 Реализация на бизнес логиката

Бизнес логиката е разположена основно в приложния слой. Там се намират операциите за управление на оферти, създаване на колекции, публикуване, обработка на запитвания и внос от външни източници.

Операторският каталог позволява създаване и поддръжка на оферти. В зависимост от начина на работа на оператора, офертите могат да бъдат въведени ръчно или групово. Това улеснява първоначалното попълване на каталога и последващото му обновяване.

My offers

New offer

Title: Country: City:

Price: Currency: Nights:

Board: Transport: Image URL:

Description:

[Bulk import from CSV](#)

Title	Country	Board	Price	Status	
Sunny Beach Weekender	BG	AllInclusive	220 EUR	Published	Edit Unpublish
Sofia Mountain Retreat	BG	HalfBoard	280 EUR	Published	Edit Unpublish
Cairo & Nile Cruise	EG	FullBoard	1,450 EUR	Published	Edit Unpublish
Sharm El Sheikh Diver	EG	AllInclusive	690 EUR	Published	Edit Unpublish

Фигура 6.3. Операторски каталог

Конекторите към външни доставчици са реализирани чрез общ интерфейс. Всеки конектор отговаря за конкретен тип връзка, но резултатът от изпълнението се свежда до общ модел. Така останалата част от системата не зависи от конкретния формат или протокол на доставчика.

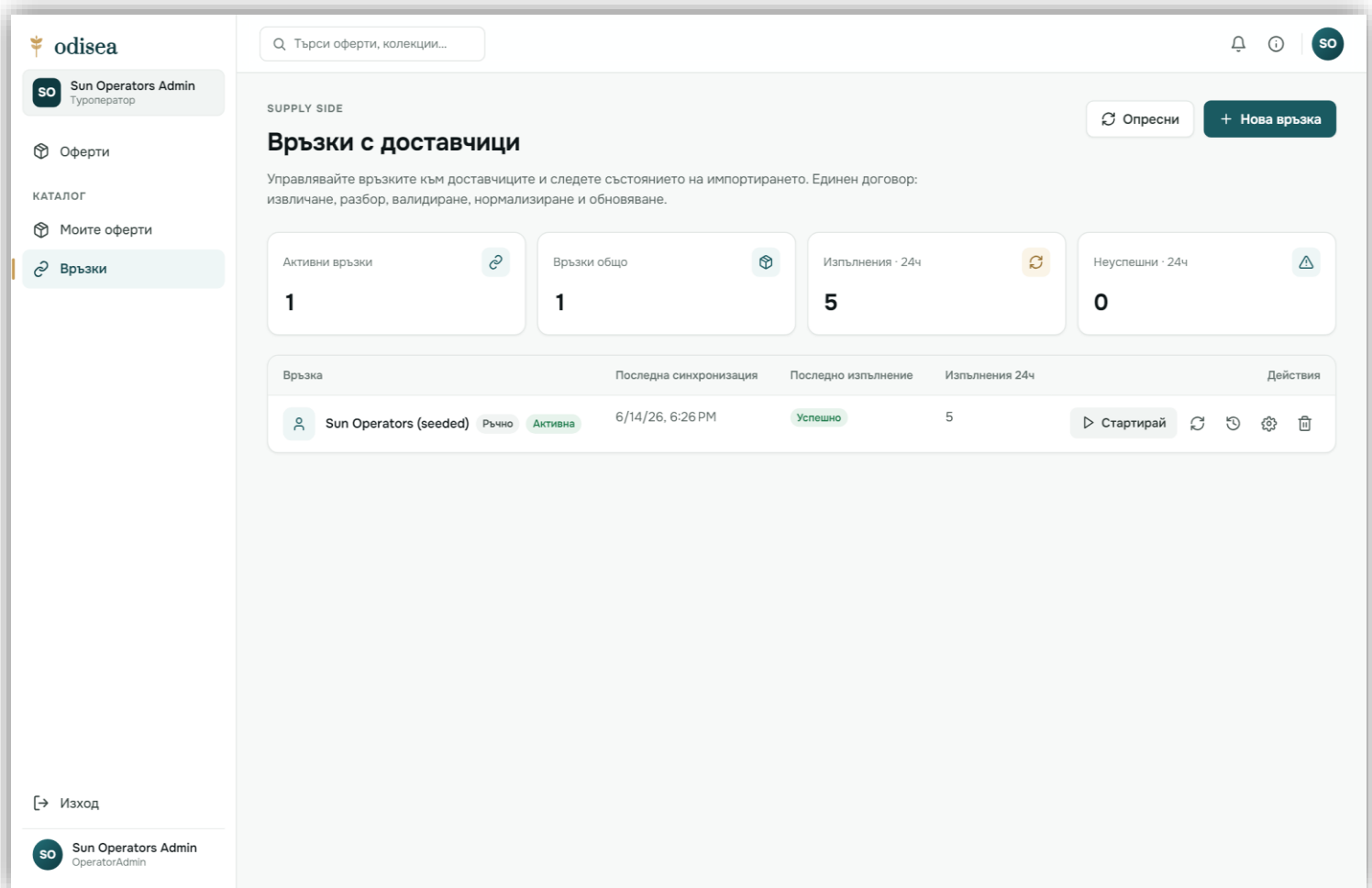
```
public interface IConnector
{
    SupplierConnectionKind Kind { get; }

    Task<ConnectorRunResult> RunAsync(
        SupplierConnection connection,
```

```
} CancellationToken ct);
```

Изпълнителят на синхронизацията избира подходящия конектор според типа на връзката. След това получените записи се валидират, нормализират и записват в каталога. Ако част от записите са невалидни, те се отбелязват като грешки, без да спират обработката на останалите.

Управлението на връзките към доставчици е отделено в собствен екран. Чрез него операторът може да добавя връзки, да стартира ръчно импортиране и да преглежда историята на изпълненията.



Фигура 6.4. Управление на връзки

Тази реализация позволява постепенно добавяне на нови доставчици, без да се променя общият процес на обработка.

6.5 Реализация на базата данни

Достъпът до данните е реализиран чрез Entity Framework Core. Всяка същност има отделна конфигурация, която описва таблицата, ключовете, връзките, ограниченията и преобразуванията на типовете.

Това прави модела по-лесен за поддръжка и позволява промените по схемата да се управляват чрез миграции.

Следният фрагмент показва примерна конфигурация на връзка към доставчик.

```
b.ToTable("supplier_connections");
b.HasKey(x => x.Id);

b.Property(x => x.Kind)
    .HasConversion<string>()
    .HasMaxLength(32);

b.Property(x => x.ConfigJson)
    .HasColumnType("jsonb")
    .IsRequired();

b.HasIndex(x => x.OperatorId);
```

Изброимите типове се записват като текст, което прави данните по-четими при директен преглед на базата. Конфигурационни стойности като настройки на връзки, филтри или изгледи могат да се съхраняват в `jsonb` колони. Така схемата остава релационна за основните връзки, но допуска гъвкавост при по-променливи структури.

Миграциите поддържат синхрон между кода и базата данни. При промяна в модела се създава нова миграция, която описва необходимите промени в схемата. Това улеснява стартирането на проекта в различни среди и намалява риска от несъответствия.

6.6 Реализация на комуникацията между модулите

Комуникацията между клиентската и сървърната част се извършва чрез REST заявки с обмен на JSON. Административният портал използва защитени крайни точки, докато вградените компоненти използват публични крайни точки, базирани на ключа на публикацията.

При зареждане на публикация компонентът първо заявява публичния манифест. Той съдържа информация за темата, изгледа и настройките, които са необходими за визуализацията. След това компонентът зарежда офертите и ги показва в сайта на агенцията.

При изпращане на запитване от турист данните се записват към съответната агенция и могат да бъдат прегледани в административния портал. Ако агенцията е настроила външна интеграция, системата може да изпрати и изходящо известие. Известията се подписват, за да може получателят да провери, че са изпратени от платформата.

Така комуникацията е разделена според типа достъп. Административната част използва потребителска идентичност и роли, а публичната част използва ключове на публикации и ограничения по домейн.

6.7 Основни алгоритми и процеси

В реализацията могат да бъдат открити три основни процеса: филтриране на оферти, добавяне на оферти от конектор и планирана синхронизация.

Филтрирането на оферти превръща зададена от потребителя спецификация в заявка към базата данни. Спецификацията може да съдържа различни условия, например дестинация, период, цена, транспорт или оператор. Условията могат да бъдат комбинирани, така че колекцията да описва точно кои оферти трябва да попаднат в публикацията.

За да се избегнат прекалено сложни заявки, системата може да ограничава броя на условията и дълбочината на логическите групи. Това предпазва сървъра от тежки заявки и прави поведението по-предвидимо.

Добавяне на оферти от конектор е реализиран така, че повторно изпълнение да не създава дубликати. Всяка оферта от външен доставчик се разпознава чрез външен идентификатор и връзка към източника. Ако офертата вече съществува, тя се обновява. Ако не съществува, се създава нов запис.

Планираната синхронизация проверява кои връзки към доставчици са активни и трябва да бъдат изпълнени. След това за всяка подходяща връзка се стартира внос. Резултатът се записва в история, за да може операторът да види кога е изпълнена синхронизацията и какъв е бил резултатът.

6.8 Обработка на грешки

Грешките се обработват по стандартизиран начин. Очакваните бизнес грешки, като липса на права, невалидни входни данни или ресурс, който не принадлежи на текущата организация, се превръщат в ясен HTTP отговор. Този отговор съдържа информация за проблема, без да разкрива вътрешни технически детайли.

При операции, които обработват много записи, се използва частичен успех. Това означава, че валидните записи се обработват, а невалидните се връщат със съобщение за грешка и позиция във входния пакет. Така една грешна оферта не проваля целия внос.

Неочакваните грешки се прихващат на централно ниво и се записват в логовете. Към клиента се връща обобщен отговор. По този начин потребителят получава разбираема информация, а разработчикът запазва възможност да проследи причината в системните записи.

6.9 Сигурност в реализацията

Сигурността е приложена на няколко места в реализацията. Първо, административните крайни точки изискват автентикация. Второ, достъпът до действия се ограничава чрез роли. Трето, ресурсите се

проверяват спрямо организацията, към която принадлежи текущият потребител.

Следният фрагмент показва типичен шаблон за проверка дали даден ресурс принадлежи на текущия оператор.

```
private async Task<SupplierConnection?> FindOwned(Guid id, CancellationToken ct)
{
    var operatorId = operatorCtx.RequireOperator();

    var conn = await db.SupplierConnections.FirstOrDefaultAsync(c => c.Id == id, ct);

    return conn is not null && conn.OperatorId == operatorId ? conn : null;
}
```

Този шаблон предотвратява достъп до ресурс, който съществува в системата, но принадлежи на друга организация. При такъв случай отговорът не трябва да разкрива дали ресурсът реално съществува. Това намалява риска от извличане на информация чрез пробване на идентификатори.

Публичните компоненти използват различен механизъм за достъп. Те не работят с потребителска сесия, а с публичен ключ на публикацията. За всяка публикация могат да бъдат зададени разрешени домейни, което ограничава сайтовете, от които тя може да бъде зареждана.

Изходящите известия към външни системи се подписват, за да може получателят да провери техния произход. Публичните входни точки, като изпращане на запитване, трябва да бъдат защитени и срещу прекомерен брой заявки.

По време на разработката са проверени крайните точки, които работят с ресурси на организации. Целта на тази проверка е да се гарантира, че достъпът се ограничава не само по роля, но и по принадлежност на ресурса. Това е особено важно за платформа, в която различни агенции и туроператори използват обща система.

Следващата глава разглежда тестването и валидацията на разработеното решение.

Глава 7: Тестване и валидация

Тестването има за цел да провери дали платформата Odisea изпълнява основните си функции по предвидим начин. Особено важно е да се потвърди, че потребителите работят само с ресурсите на своята организация, че вносът на оферти не създава дубликати и че основните клиентски сценарии протичат коректно.

Настоящата глава разглежда подхода към тестването, модулните и интеграционните проверки, ръчната валидация на основните сценарии, ограниченията на проведеното тестване и изводите от резултатите.

7.1 Подход към тестването

Подходът към тестването е насочен към проверка на критичните части на системата, а не само към формално покритие на кода. Приоритет имат онези сценарии, при които грешка би довела до реален проблем: неправилен достъп до чужди ресурси, некоректно филтриране на оферти, дублиране при повторен внос, неуспешна автентикация или грешно поведение на клиентския интерфейс.

Тестовите са разделени на няколко групи. Модулните тестове проверяват отделни правила и услуги. Интеграционните тестове проверяват взаимодействието между контролери, приложна логика и контекст на базата. Клиентските тестове проверяват основното поведение на страниците и услугите в портала. Допълнително са преминати ръчно основните потребителски сценарии в локална контейнеризирана среда.

7.2 Модулни тестове

Модулните тестове покриват основните правила в приложния слой и поведението на части от сървърния интерфейс. Проверяват се филтрирането на оферти, обработката на колекции, работата на конекторите, автентикацията и ограничаването на достъпа според организацията на потребителя.

При филтрирането се проверяват както валидни условия, така и невалидни входни данни. Целта е да се потвърди, че системата избира правилните оферти и отказва некоректни спецификации. При конекторите се проверява дали повторното изпълнение на един и същ внос обновява вече съществуващи оферти, вместо да създава дубликати.

Особено внимание е отделено на достъпа до ресурси. Потребителят трябва да вижда и променя само данните на организацията, към която принадлежи. Следният фрагмент показва примерен тест, който проверява дали оператор вижда само собствените си връзки към доставчици.

```
[Fact]
public async Task List_returnsOnlyCallingOperatorsConnections()
{
    await using var db = NewDb();

    var mine = Guid.NewGuid();

    await SeedConnection(db, mine, "Mine");
    await SeedConnection(db, Guid.NewGuid(), "Theirs");

    var result = await ControllerFor(db, mine).List(default);

    var ok = Assert.IsType<OkObjectResult>(result);
    var dtos = Assert.IsAssignableFrom<IEnumerable<SupplierConnectionDto>>(ok.Value);

    Assert.Single(dtos);
}
```

Този тип тестове са важни, защото проверяват не само ролята на потребителя, но и принадлежността на конкретния ресурс. Така се намалява рискът една агенция или един оператор да получи достъп до данни на друга организация.

7.3 Интеграционни тестове

Интеграционните тестове проверяват дали няколко части на системата работят коректно заедно. В сървърната част това включва контролери, контекст на базата, регистрирани услуги и част от приложната логика.

Проверява се дали заявките към контролерите връщат очакваните резултати, дали валидацията работи правилно и дали кодовете на отговорите са подходящи при успешни и неуспешни операции. Например при опит за достъп до ресурс на друга организация системата трябва да върне отговор, който не позволява извличане на информация за съществуването на този ресурс.

Отделно се проверява регистрацията на зависимостите. Тази проверка показва дали приложението може да бъде сглобено коректно при стартиране и дали липсващи услуги или грешни зависимости няма да доведат до проблем още при първоначалното зареждане.

Фоновото планиране също е обхванато от тестове. Проверява се дали за синхронизация се избират само активни връзки, които са готови за изпълнение. Това е важно, защото грешка в тази логика може да доведе до твърде чести заявки към доставчици или до пропуснати обновявания.

7.4 Проверка на основните потребителски сценарии

Освен автоматизираните тестове са преминати и основните потребителски сценарии. Те потвърждават, че отделните части на платформата работят заедно в реалистичен поток.

Първият сценарий е регистрация и вход в системата. Проверява се дали потребителят може да създаде профил, да влезе в портала и да получи достъп до функционалности според ролята си.

Вторият сценарий е работа на туроператор. Проверява се създаване на оферти, групово въвеждане и преглед на операторския каталог. При връзките към доставчици се проверява създаване на връзка, ръчно стартиране на внос и преглед на резултатите.

Третият сценарий е работа на агенция. Проверява се създаване на колекция, избор на изглед, настройване на тема и създаване на публикация. След това се проверява дали публикацията може да бъде заредена чрез вграден компонент.

Четвъртият сценарий е изпращане на запитване от турист. Проверява се дали формата във вградения компонент приема данните, дали запитването се записва към правилната агенция и дали може да бъде прегледано в административния портал.

Обобщение на покритите области е представено в таблица 7.1.

Таблица 7.1. Обхват на тестването по области

Област на тестване	Примерни проверки
Автентикация и регистрация	вход, регистрация, поддържане на сесия
Каталог и филтри	условия за филтриране, вложени групи, обработка на колекции
Изоляция между организации	достъп само до собствени ресурси, отказ при чужд ресурс
Конектори	повторен внос без дублиране, готовност за синхронизация, избор на адаптер
Контролери	създаване, редактиране, валидация, кодове на отговор
Портал	логика на страници, извикване на заявки, обработка на грешки
Вградими компоненти	зареждане на публикация, визуализиране на оферти, изпращане на запитване

7.5 Тестване на производителност

В настоящата версия не са проведени пълни количествени натоварващи тестове. Основният фокус е поставен върху функционалната коректност, сигурността на достъпа и стабилността на основните процеси.

Въпреки това при реализацията са взети предвид няколко предпоставки за добра производителност. Публичният манифест на публикацията може да се кешира. Заявките към базата използват филтриране по основни идентификатори и индексирани полета. Потребителските операции, като синхронизация с доставчици, се изпълняват отделно от основния потребителски поток.

Натоварващото тестване остава следваща стъпка. То трябва да включва измерване на време за зареждане на публикация, време за

обработка на голям каталог, поведение при множество едновременни заявки и устойчивост при бавен външен доставчик.

7.6 Анализ на резултатите

Проведените тестове показват, че основните сценарии на платформата работят коректно. Успешно са проверени входът в системата, управлението на оферти, създаването на публикации, зареждането чрез вграден компонент и обработката на запитвания.

Най-голяма практическа стойност имат тестовете за изолация между организациите. Те помагат да се потвърди, че проверката на достъпа не се изчерпва само с ролята на потребителя, а включва и принадлежността на ресурса. Подобни тестове остават важна защита срещу бъдещи промени, които биха могли неволно да нарушат това правило.

Полезни са и тестовете за конекторите, защото процесът по внос работи с данни от външни източници, които могат да бъдат непълни или невалидни. Проверката за частичен успех и повторен внос намалява риска от загуба на валидни данни или създаване на дубликати.

7.7 Ограничения на проведеното тестване

Проведеното тестване има няколко ограничения. Не са реализирани автоматизирани тестове от край до край през реален браузър. Такива тестове биха позволили по-пълна проверка на целия поток от вход в портала до зареждане на публикация в сайт на агенция.

Не са проведени и количествени натоварващи тестове. Поради това не могат да бъдат посочени точни стойности за максимален брой едновременни заявки, време за отговор при голям каталог или поведение при продължителна синхронизация.

Конекторите са проверени основно със синтетични или контролирани входни данни. При реални доставчици могат да възникнат допълнителни случаи, свързани с непълен формат, различно кодиране, липсващи полета или неочаквани промени в структурата на данните.

Тези ограничения не обезсилват проведените проверки, но очертават посоката за бъдещо развитие на тестовия процес. Следваща стъпка е добавяне на тестове от край до край, натоварващи тестове и проверки с реални доставчици. Следващата глава описва инсталацията, стартирането и използването на платформата.

Глава 8: Ръководство за инсталация и използване

Настоящата глава описва необходимите стъпки за стартиране и използване на платформата Odisea. Целта е работата със системата да бъде възпроизводима както в среда за разработка, така и при контейнеризирано стартиране. Описани са основните зависимости, стартирането на клиентската и сървърната част, конфигурацията на базата данни и основните действия на потребителите.

8.1 Необходими зависимости

За стартиране на системата чрез контейнери е необходим Docker с Docker Compose. Този подход е най-лесен, защото базата данни, сървърният интерфейс и клиентската част се стартират като предварително описани услуги.

За локална разработка са необходими и следните инструменти:

Таблица 8.1. Основни зависимости за стартиране на проекта

Инструмент	Предназначение
.NET SDK	Стартиране и разработка на сървърната част
Node.js	Стартиране и изграждане на клиентските приложения
Git	Управление на версиите и работа с хранилището
Docker	Стартиране на инфраструктурата в контейнери
Docker Compose	Оркестрация на услугите

При контейнеризирано стартиране PostgreSQL не е необходимо да бъде инсталиран отделно, тъй като се стартира като самостоятелна услуга чрез Docker Compose.

8.2 Стартиране на клиентската част

Клиентската част включва административния портал, публичния сайт и библиотеката с вградими компоненти. В режим на разработка порталът се стартира чрез Node.js и локален development server. Това позволява автоматично презареждане при промени в кода и улеснява ежедневната разработка.

Примерна последователност за стартиране на портала е:

```
cd frontend/portal
npm install
npm run start
```

Библиотеката с вградими компоненти се намира в отделна директория и се компилира до самостоятелен пакет. Този пакет може да бъде използван в сайтовете на агенциите чрез предоставения код за вграждане.

```
cd frontend/components
npm install
npm run build
```

При контейнеризирано стартиране клиентската част се изгражда предварително и се отдава от уеб сървър. Това доближава поведението до реална среда за внедряване.

8.3 Стартиране на сървърната част

Сървърната част се намира в директорията `backend`. В режим на разработка тя може да бъде стартирана чрез инструментите на .NET.

```
cd backend/src/Odisea.WebAPI
dotnet run
```

При първо стартиране се прилагат миграциите към базата данни и се зарежда начална конфигурация, ако такава е предвидена в проекта. Това осигурява необходимата структура на таблиците и позволява системата да започне работа без ръчно създаване на базата.

При стартиране чрез Docker тези действия се изпълняват в контейнера на сървърния интерфейс. По този начин локалната настройка се свежда до стартиране на контейнерите.

8.4 Конфигурация на базата данни

Системата използва PostgreSQL като основна база данни. При контейнеризирано стартиране базата работи като отделна услуга и използва собствен Docker том за съхранение на данните.

В конфигурацията базата е публикувана на порт `5433`, за да не влиза в конфликт с локална PostgreSQL инсталация, която често използва стандартния порт `5432`.

Примерни стойности за локална среда са:

Таблица 8.2. Примерна конфигурация на базата данни

Параметър	Стойност
Host	localhost
Port	5433
Database	odisea
User	postgres
Password	задава се в конфигурацията

Промените по схемата се управляват чрез миграции. Това позволява структурата на базата да се развива заедно с кода и намалява риска от несъответствия между различни среди.

8.5 Стартиране с Docker

Най-лесният начин за стартиране на цялата система е чрез Docker Compose от основната директория на проекта.

```
docker compose up -d --build
```

Командата изгражда необходимите образи и стартира основните услуги: базата данни, сървърния интерфейс и клиентската част. След успешно стартиране системата е достъпна на следните адреси:

Таблица 8.3. Локални адреси на услугите

Услуга	Адрес
Портал и публичен сайт	http://localhost:4200
Сървърен интерфейс	http://localhost:8080
PostgreSQL	localhost:5433

Спирането на услугите се извършва със следната команда:

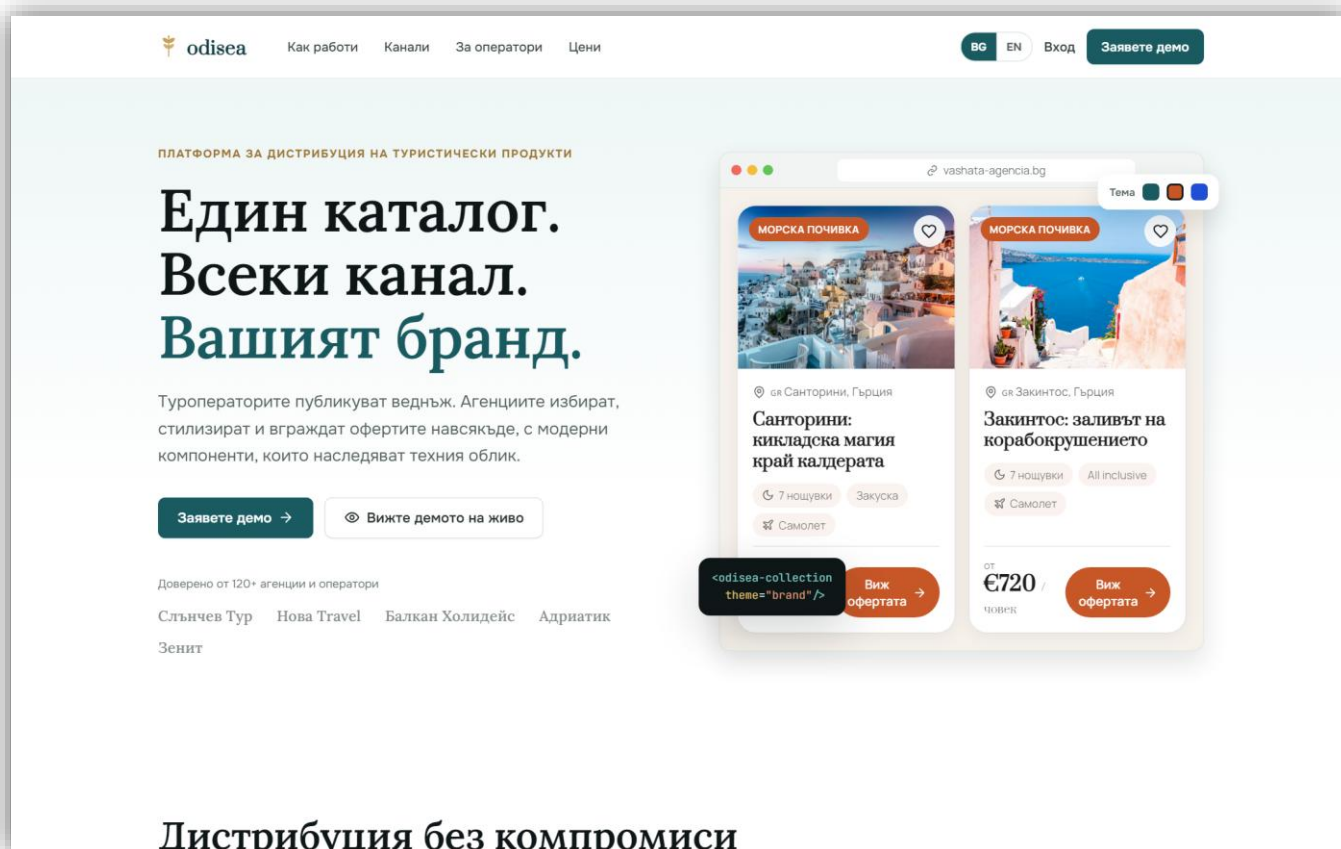
```
docker compose down
```

Ако е необходимо системата да бъде върната в чисто състояние, може да се премахне и томът на базата данни. Това изтрива локално съхранените данни, затова трябва да се използва внимателно.

```
docker compose down -v
```

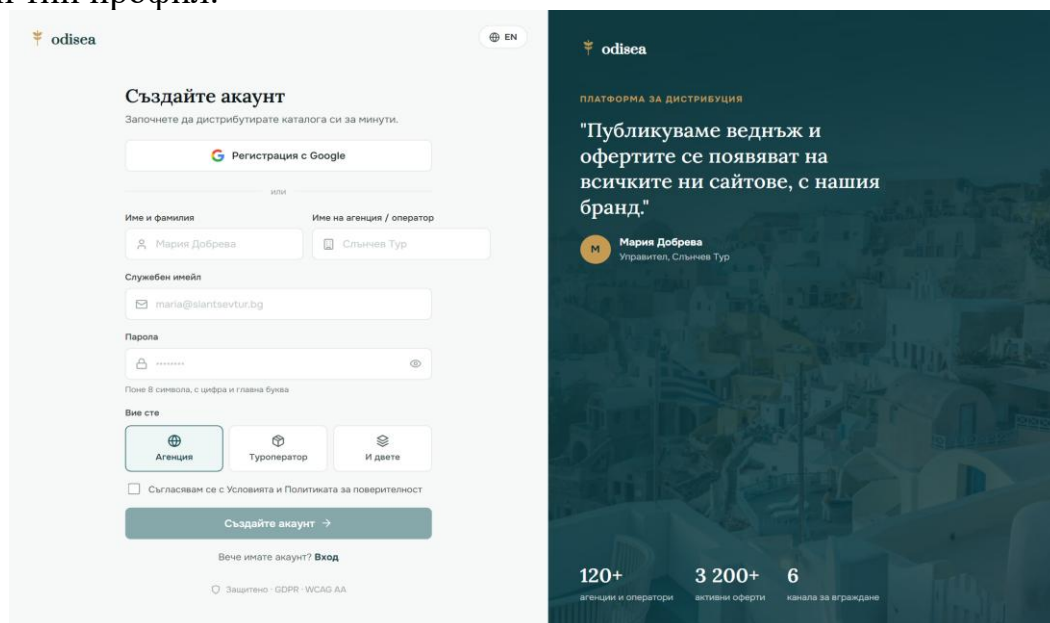
8.6 Основни стъпки за използване

Работата с платформата започва от публичния сайт. Той представя основната идея на Odisea и насочва потребителя към регистрация или вход в системата.



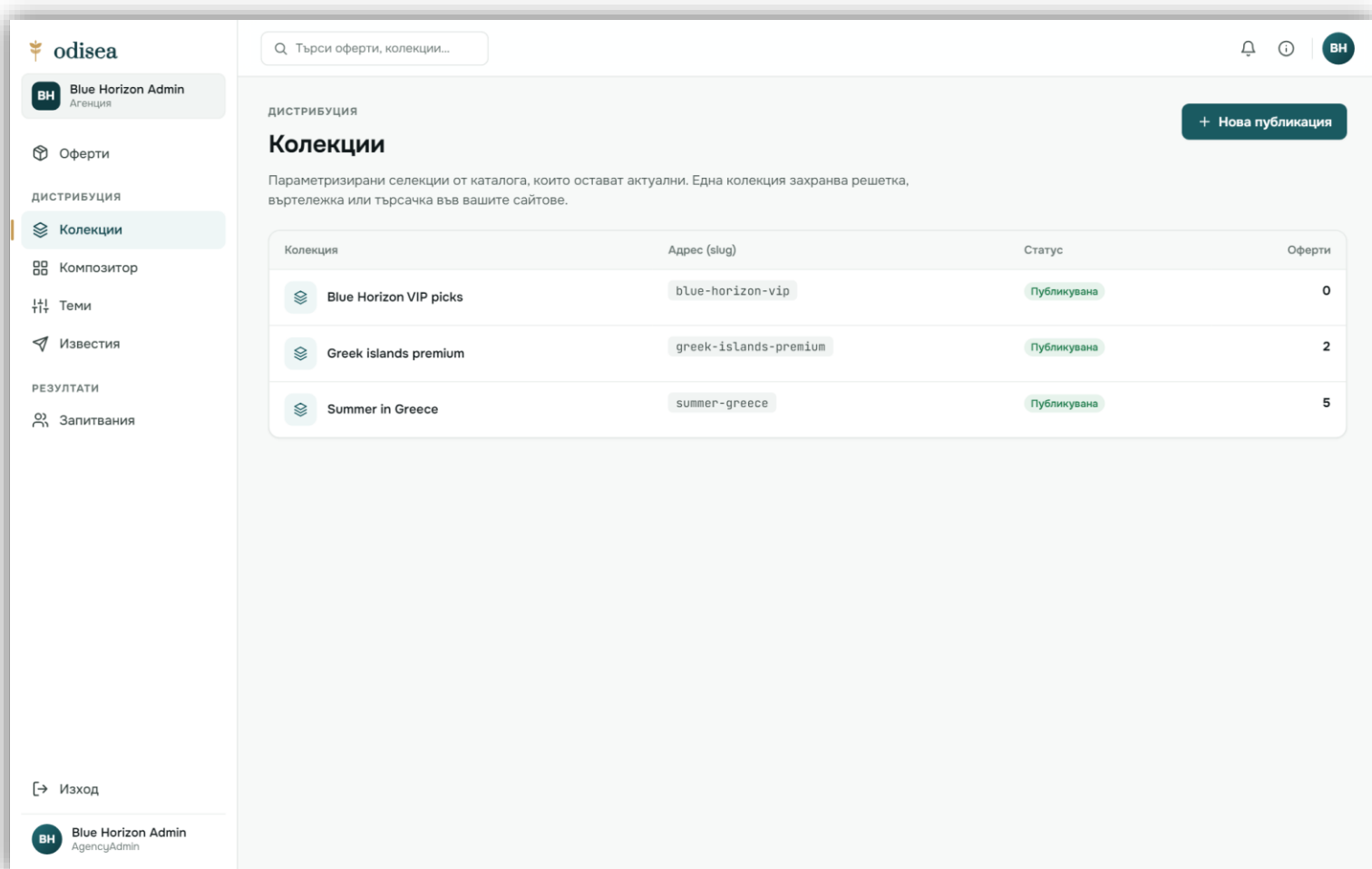
Фигура 8.1. Начална страница на платформата

Нов потребител създава акаунт през регистрационната форма. При регистрацията се посочват основни данни за потребителя и организацията, например име, електронна поща, име на агенция или туроператор и избран тип профил.



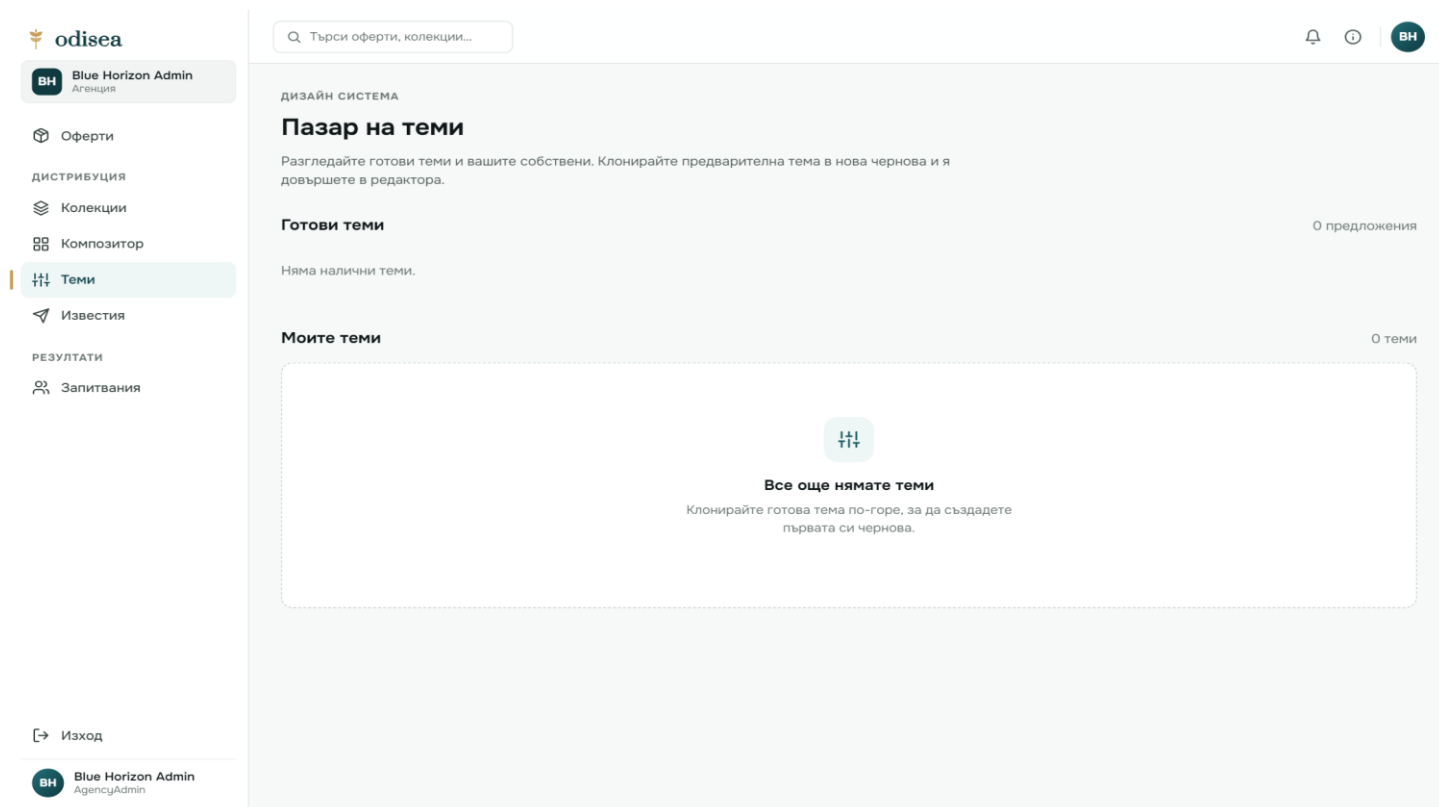
Фигура 8.2. Регистрация на нов акаунт

След вход в системата действията зависят от ролята на потребителя. Туроператорът управлява оферти, като може да ги въвежда ръчно, групово или чрез връзка към външен източник. Агенцията работи с наличния каталог и създава колекции от оферти, които да публикува в своя сайт.



Фигура 8.3. Управление на колекции

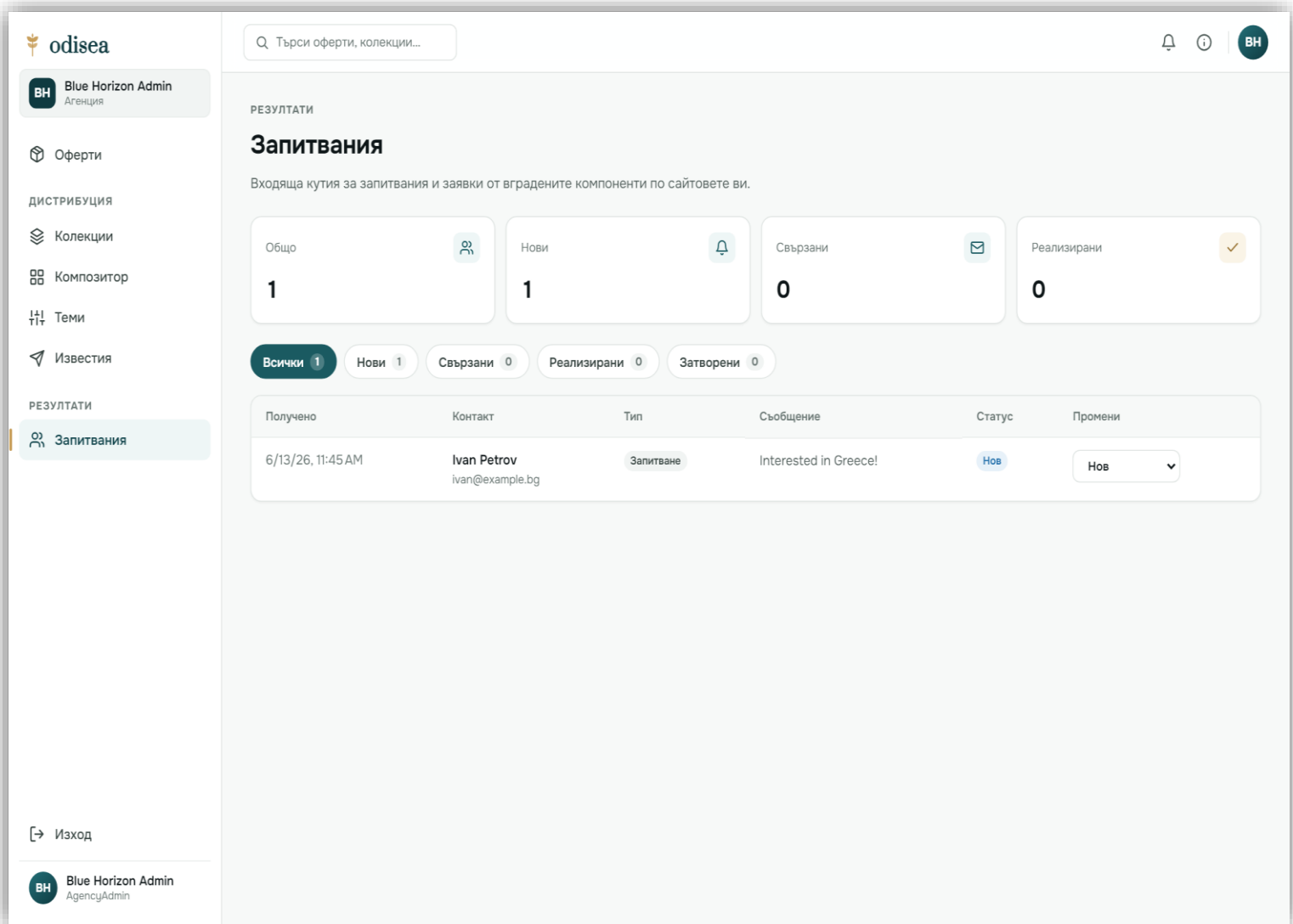
Визуалната идентичност на публикуваните оферти се настройва чрез теми. Агенцията може да избере тема и да я адаптира спрямо своя сайт, без да променя кода на вградимите компоненти.



Фигура 8.4. Пазар на теми

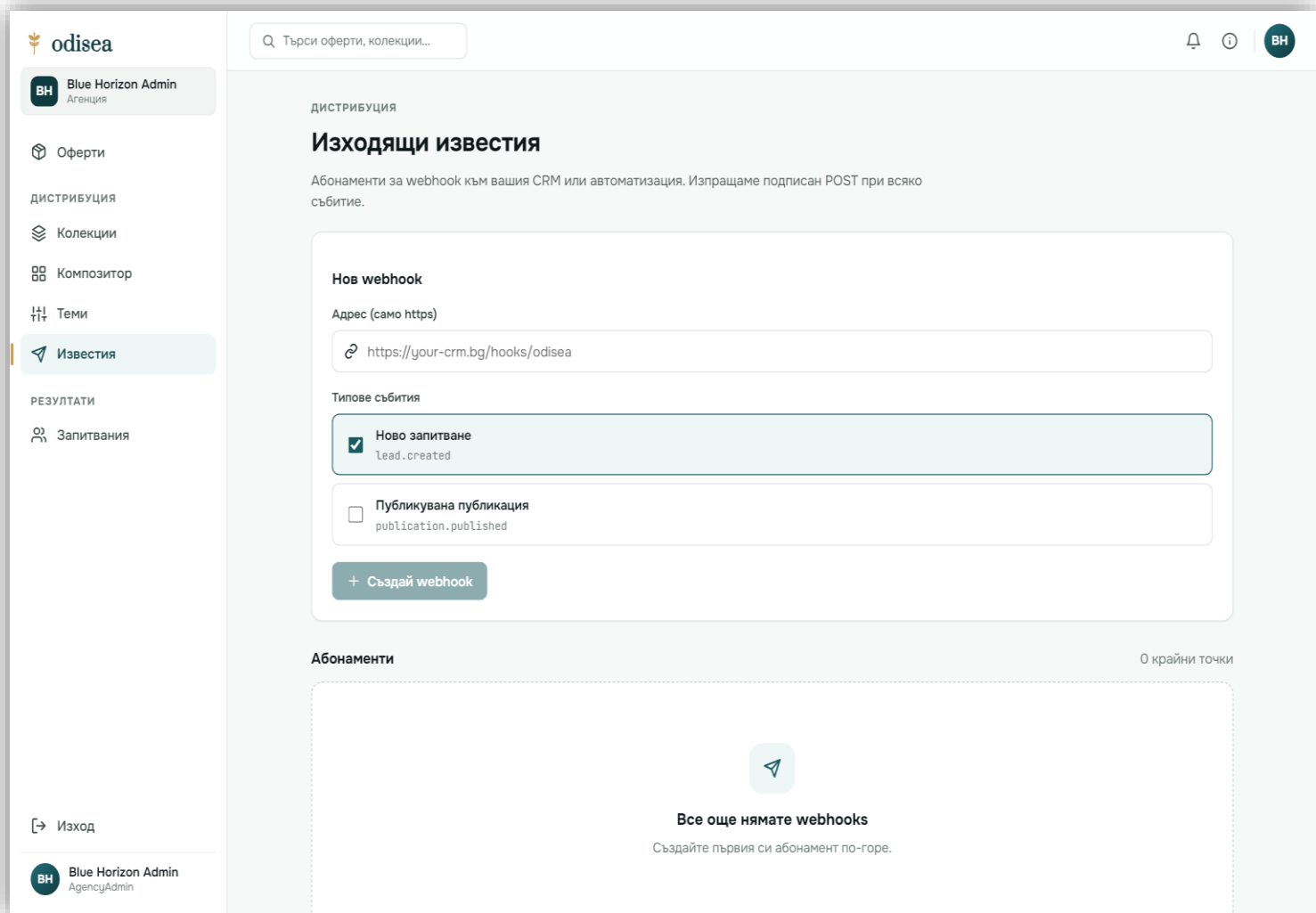
След като колекцията, изгледът и темата са избрани, агенцията създава публикация. Системата генерира код за вграждане, който се поставя в сайта на агенцията. Когато посетител отвори страницата, офертите се зареждат чрез вградимите компоненти.

При интерес от страна на туриста се изпраща запитване. То постъпва в обща кутия в административния портал, където агенцията може да го прегледа и обработи.



Фигура 8.5. Кутия със запитвания

За връзка с външни системи агенцията може да регистрира изходящи известия. Те се използват при събития като ново запитване и позволяват интеграция с други инструменти.



Фигура 8.6. Управление на изходящи известия

8.7 Административна част

Администраторът на платформата има достъп до функции извън рамките на отделна организация. Той може да наблюдава регистрираните организации, да управлява общи настройки, да преглежда състоянието на системата и да подпомага поддръжката при възникване на проблеми.

Тази роля се използва внимателно, тъй като има по-широк достъп от ролите на агенциите и туроператорите. Основната ѝ цел е техническо и организационно управление на платформата, а не ежедневна работа с оферти и публикации.

Описаните стъпки показват как системата може да бъде стартирана, конфигурирана и използвана в основните си сценарии. Следващата глава прави оценка на постигнатото и очертава възможностите за бъдещо развитие.

Глава 9: Оценка на решението и бъдещо развитие

Настоящата глава обобщава резултатите от разработката на платформата Odisea. Оценката е направена спрямо поставените цели, реализираните функционалности и ограниченията на текущата версия. В края са посочени възможности за бъдещо развитие, които следват естествено от настоящия обхват на проекта.

9.1 Постигнати резултати

В резултат на разработката е изградена работеща платформа за управление, публикуване и вграждане на туристически оферти в сайтове на агенции. Реализиран е основният поток, при който туроператорът въвежда или внася оферти, агенцията ги подбира в колекция, настройва визуално представяне и създава публикация, която може да бъде заредена чрез вграден компонент.

Системата поддържа основните понятия, необходими за този процес: оферти, колекции, теми, изгледи, публикации, връзки към доставчици и запитвания. Въведено е ясно разделение между съдържанието и начина на представяне, което позволява една и съща оферта да бъде използвана в различни публикации и в различен визуален контекст.

Реализиран е административен портал за работа на агенции и туроператори. Чрез него могат да се управляват оферти, колекции, теми, публикации, връзки към доставчици и постъпили запитвания. Разработени са и вградими компоненти, които зареждат публична публикация и я визуализират в сайта на агенцията.

От техническа гледна точка проектът включва сървърна част, клиентска част, база данни, контейнеризирано стартиране и автоматизирани тестове. Това позволява системата да бъде стартирана повторно и да бъде проверявана при бъдещи промени.

9.2 Сравнение с поставените цели

Поставените в началото цели са изпълнени в основната си част. Разработката доказва, че избраният подход с отделяне на каталог, колекции, теми, изгледи и публикации е приложим за проблема с разпространението на туристически оферти.

Таблица 9.1. Съпоставка между поставените цели и реализираните резултати

Цел	Степен на изпълнение
Централизирано управление на туристически оферти	Реализирано чрез каталог и операторски модул
Подбор на оферти от агенции	Реализирано чрез колекции и филтри
Визуално настройване според марката на агенцията	Реализирано чрез теми и дизайн токени
Публикуване в сайт на агенция	Реализирано чрез публикации и публичен ключ
Вграждане без iframe	Реализирано чрез уеб компоненти
Внос от външни източници	Реализиран начален механизъм чрез конектори
Приемане на запитвания	Реализирано чрез публична форма и кутия със запитвания
Пълен резервационен процес	Оставен извън текущия обхват

Сравнението показва, че основната цел на проекта е постигната на ниво функционален прототип. Платформата позволява управление и публикуване на оферти, както и тяхното визуализиране в сайт на агенция. Функции като пълна резервация, плащания, външна автентикация и готови приставки за популярни платформи са оставени за бъдещо развитие, тъй като не са критични за доказване на основния модел.

9.3 Предимства на разработеното решение

Основното предимство на Odisea е, че намалява компромиса между актуалност на съдържанието и визуално вписване в сайта на агенцията. При ръчно поддържане агенцията има контрол върху изгледа, но трудно поддържа актуалност. При iframe решенията актуалността е по-добра, но визуалната интеграция е ограничена. Разработеното решение съчетава централизирано съдържание с вградими компоненти, които могат да приемат облика на приемащия сайт.

Друго предимство е ясното разделение между основните понятия. Офертите, колекциите, темите, изгледите и публикациите имат отделни роли в системата. Това улеснява поддръжката и позволява развитие на отделни части без ненужно засягане на останалите.

Решението е подходящо и за работа със множество организации. Всяка агенция или туроператор работи в собствен организационен контекст, а достъпът до ресурси се проверява както по роля, така и по принадлежност на данните. Това е важно условие за платформа, която обслужва повече от една организация в обща среда.

Предимство е и възможността за постепенно развитие. Модулната структура позволява добавяне на нови конектори, нови типове изгледи, допълнителни канали за публикуване и разширени механизми за проследяване, без да се променя основният модел на системата.

9.4 Ограничения

Ограниченията на текущата версия са свързани основно с обхвата и етапа на развитие на проекта. Платформата реализира основния процес по управление, публикуване и вграждане на оферти, но не покрива пълен резервационен цикъл. Това означава, че заявките от туристи се приемат като запитвания, а не като завършени резервации с плащане, потвърждение и издаване на документи.

Схемата на офертата е разработена като начална версия. Тя покрива основните данни, необходими за представяне на туристически продукт, но трябва да бъде проверена с реални емисии от различни доставчици. При такава проверка е възможно да се появят допълнителни полета, специфични формати или изключения, които да наложат разширяване на модела.

Конекторите са реализирани като механизъм, но не са изчерпани всички възможни типове външни източници. В реална среда доставчиците могат да използват различни структури, различно качество на данните и различни правила за обновяване. Това изисква допълнителна работа по адаптери, диагностика и устойчивост при грешки.

Не са проведени пълни натоварващи тестове и автоматизирани тестове от край до край през реален браузър. Поради това текущата оценка на производителността е ограничена до архитектурни предпоставки и функционални проверки, без точни количествени резултати при голямо натоварване.

9.5 Възможности за бъдещо развитие

Първата посока за бъдещо развитие е валидирането на модела на офертата с реални доставчици. Това ще покаже дали текущата структура е достатъчна и кои полета трябва да бъдат разширени. На тази основа могат да се разработят по-зрели конектори към реални източници на туристически данни.

Втората посока е усъвършенстване на механизма за синхронизация. Към него могат да бъдат добавени повторни опити при временна грешка, по-гъвкаво планиране, по-подробна история на изпълненията и по-добра диагностика за невалидни записи. Това ще направи вноса по-надежден при работа с външни системи.

Трета възможност е добавяне на готови канали за публикуване. Освен универсален код за вграждане, платформата може да предложи приставки за системи като WordPress или други популярни платформи за сайтове. Това би улеснило агенции, които не разполагат със собствен технически екип.

Четвърта посока е развитие на запитванията към по-структуриран резервационен процес. Това може да включва избор на дата, брой

туристи, допълнителни услуги, потвърждение от агенция и последваща интеграция с платежни или резервационни системи. Подобно разширение трябва да бъде реализирано внимателно, тъй като въвежда повече бизнес правила и по-високи изисквания към надеждността.

Петата посока е разширяване на тестването. Необходимо е да се добавят автоматизирани тестове през реален браузър, натоварващи тестове и проверки с реални доставчици. Това ще даде по-точна представа за поведението на системата при реална употреба.

В обобщение, разработената платформа изпълнява основната си цел и доказва приложимостта на избрания подход. Текущата версия поставя стабилна основа за бъдещо развитие, като запазва ясен модел, контрол върху достъпа и възможност за постепенно надграждане.

Приложение – Изходен код

Пълният изходен код на платформата се поддържа в система за управление на версии (GitHub) и е организиран според описаната в Глава 6 структура. Сървърната част е разделена на проекти по слоевете на чистата архитектура, а клиентската съдържа портала и библиотеката с вградими компоненти. Структурата на хранилището позволява стартиране на цялата система с една команда, както е описано в Глава 8.

Можете да получите достъп до кода чрез следния линк:

<https://github.com/odisea-network/odisea>

ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа разгледа проектирането и реализацията на платформа за управление и агрегиране на вградими уеб компоненти за нуждите на туризма. Изходната точка беше реален проблем: между туроператорите, които създават офертите, и агенциите, които ги продават, липсва обща технологична среда, а наследените начини за вграждане поставят агенциите пред избор между актуалност и собствен облик.

Работата показва, че този компромис може да бъде премахнат. Чрез вградими компоненти със Shadow DOM и тематизиране през токени едно и също съдържание се представя в облика на различни агенции, докато централно поддържаният каталог запазва актуалността му. Около тази идея беше изградена цялостна архитектура с ясно разделение на съдържание, селекция, представяне и разпространение, реализирана по принципите на чистата архитектура и описана чрез утвърдени архитектурни изгледи.

Освен представянето беше решен и въпросът с предлагането. Механизмът за конектори с общ договор и заменяеми адаптери превръща постъпването на оферти от поредица отделни интеграции в единна разширяема способност, а планировчикът и проверката на актуалността поддържат каталога надежден. Многонаемната изолация беше заложена последователно в реализацията и потвърдена с тестове, което прави решението приложимо за среда с много независими организации.

Постигнатото покрива поставените цели на ниво работещо решение, а ясно отбелязаните ограничения и посоки за бъдещо развитие очертават естественото продължение на проекта. Така платформата представлява не само завършена разработка, но и основа, върху която може да се изгражда по-нататъшна функционалност, без да се нарушават установените граници.



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

Факултет Приложна математика и информатика

ДЕКЛАРАЦИЯ ЗА АВТОРСТВО НА ДИПЛОМНА РАБОТА

Долуподписаният: Аркан Ахмедов Ахмедов

специалност : Информатика и софтуерни науки фак. №
791325012

ДЕКЛАРИРАМ:

Представената от мен дипломна работа на тема: **Платформа за управление и агрегиране на вградими уеб компоненти за разработване на уеб приложения за туризма**

е лична моя авторска разработка, резултат от собствени изследвания.

Потвърждавам, че тя в нейната цялост и отделни части не е била използвана за придобиване на образователна и/или научна степен в ТУ - София или в други университети.

Формулировки, идеи и текстове, взети от други източници, са цитирани точно и с коректно посочване на техните автори. Дипломната работа не е публикувана на друго място.

Декларирам, че предоставям правото на Факултет Приложна математика и информатика при ТУ - София, съгласно процедурите и правилниците на университета, да архивира и съхранява тази дипломна работа с цел доказване във времето на моето авторство.

Дата:
София

Дипломант:

ИЗПОЛЗВАНА ЛИТЕРАТУРА

- [1] MDN Web Docs. Web Components. Mozilla.
Достъпно на: https://developer.mozilla.org/en-US/docs/Web/API/Web_components
- [2] MDN Web Docs. The Inline Frame element (iframe). Mozilla.
Достъпно на: <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe>
- [3] Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- [4] Evans, E. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003.
- [5] Brown, S. The C4 model for visualising software architecture.
Достъпно на: <https://c4model.com>
- [6] Kruchten, P. Architectural Blueprints: The 4+1 View Model of Software Architecture. IEEE Software, 12(6), 1995.
- [7] Microsoft. .NET documentation.
Достъпно на: <https://learn.microsoft.com/dotnet>
- [8] Microsoft. ASP.NET Core documentation.
Достъпно на: <https://learn.microsoft.com/aspnet/core>
- [9] Microsoft. Entity Framework Core documentation.
Достъпно на: <https://learn.microsoft.com/ef/core>
- [10] Angular. Angular documentation.
Достъпно на: <https://angular.dev>
- [11] Lit. Lit documentation.
Достъпно на: <https://lit.dev>
- [12] PostgreSQL Global Development Group. PostgreSQL 17 Documentation.
Достъпно на: <https://www.postgresql.org/docs/17/>
- [13] Docker Inc. Docker documentation.
Достъпно на: <https://docs.docker.com>
- [14] Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT). RFC 7519, IETF, 2015.
- [15] Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures. PhD dissertation, University of California, Irvine, 2000.
- [16] Nottingham, M., Wilde, E., Dalal, S. Problem Details for HTTP APIs. RFC 9457, IETF, 2023.